UMEÅ UNIVERSITY

Performance Overhead Of OpenTelemetry Sampling Methods In A Cloud Infrastructure

Tahir Mert Karkan

Master Thesis, 30 credits

Master Of Science Programme In Computing Science

2024

Abstract

This thesis explores the overhead of distributed tracing in OpenTelemetry, using different sampling strategies, in a cloud environment. Distributed tracing is telemetry data that allows developers to analyse causal events in a system with temporal information. This comes at the cost of overhead, in terms of CPU, memory and network usage, as the telemetry data has to be generated and sent through collectors that handle traces and at last sends them to a backend. By sampling using three different sampling strategies, head and tail based sampling and a mixture of those two, overhead can be reduced at the price of losing some information. To gain a measure of how this information loss impacts application performance, synthetic error messages are introduced in traces and used to gauge how many traces with errors the sampling strategies can detect. All three sampling strategies were compared for services that sent more and less data between nodes in Kubernetes. The experiments were also tested in a two and four nodes setup. This thesis was conducted with Nasdaq as it is of their interest to have high performing monitoring tools and their systems were analysed and emulated for relevance. The thesis concluded that tail based sampling had the highest overhead (71.33% CPU, 23.7% memory and 5.6% network average overhead compared to head based sampling) for the benefit of capturing all the errors. Head based sampling had the least overhead, except in the node that had deployed Jaeger as the backend for traces, where its higher total sampling rate added on average 12.75% CPU overhead for the four node setup compared to mixed sampling. Although, mixed sampling captured more errors. When measuring the overall time taken for the experiments, the highest impact could be observed when more requests had to be sent between nodes.

Acknowledgements

My greatest gratitude to family and friends that have supported me through my academic career.

Thanks to my classmates for all the years together, the ups and downs would have not been the same without you all. Special thanks to Elias for being a great friend to work with side by side on the group projects.

Last but not least, I want to thank Per-Olov Östberg for being a supportive supervisor at the University and Johan Burström for all the support at Nasdaq.

Contents

List of Tables

List of Figures

1 Introduction

This research explores how distributed tracing affects the overhead of a cloud system with different service deployments and sampling strategies. Distributed tracing is an observability technique to see how chains of events are connected. A chain of events, or a single event, is called a trace, which consists of one or more spans. A span is an individual event in a trace. As systems start to grow and become more complex, monitoring tools become more vital for management of such systems. Cloud infrastructure tends to be a complex environment that require tools such as distributed tracing, thus it is of interest to explore the dynamic of distributed tracing deployed in the cloud. This research also compares different sampling strategies for distributed tracing to see if it is possible to reduce any overhead that may come with distributed tracing. The cloud orchestration tool used for this study is Kubernetes, which is an open-source scaling and management tool used for automatic software deployment at scale.

Applications at large scale tend to require more resources, such as CPU and memory, and cloud infrastructure is a solution to be able to handle such demands at scale. It enables both vertical and horizontal scaling. When planning large cloud infrastructure, it is of importance to consider placements of nodes (computers/virtual machines) that will be connected to each other. It is also important to consider the varying amount of resources that are available on the nodes, thus placement of services becomes an important factor for performance gain. This can be a complicated task and as the system is put in place, it can be difficult to identify bottlenecks. Distributed tracing can show how different services communicate with each other and the duration of the communication, thus giving a better insight of the system. As services can become intertwined, such tools may be critical for companies to get a better overview of their systems. One aspect to consider when implementing distributed tracing is that it may in turn come with additional overhead. When a trace is generated, each span in a trace is sent to a collector that gathers them and later sends them to a back-end. The communication between services and collectors have an overhead worth considering, but also the amount of traces that may be generated and stored as distributed tracing is put in production. The cost comes at the benefit of having a system that is observable.

Sampling is one strategy to lower the overhead of distributed tracing. By choosing which traces to drop, fewer messages are sent within the system, thus lowering some

of the overhead. Additionally, fewer traces have to be stored, reducing the amount of storage capacity that a system might require over time. Some sampling techniques are head based sampling and tail based sampling. Head based sampling makes an early decision in the tracing process whether to sample a trace or not, and the decision is made by a probabilistic value. Tail based sampling requires all spans to be gathered before making a sampling decision based on the attributes of a specific trace. Different sampling strategies have their own benefits, but also come with different forms of overhead that has to be considered. These sampling strategies are explained more in depth in Section 3.6.

OpenTelemetry is a monitoring tool that allows distributed tracing for cloud infrastructure. It is a standardised way of receiving and sending telemetry data between services in a system, and also gives the ability to sample traces using both head and tail based sampling. OpenTelemetry is the tool that is used for this research when analysing the overhead of different sampling strategies and is explained more in depth in Section 3.5.

1.1 Purpose

The purpose of this thesis project is to investigate the performance overhead of distributed tracing, using OpenTelemetry. The focus is mainly on how different sampling techniques impact the performance of a system during distributed tracing. The performance is determined by measuring CPU and memory usage of the distributed system in a Kubernetes cloud environment. Additionally, network traffic and error detection will be measured as well to determine the performance of the system.

Since a large cloud infrastructure tends to have many services that communicate with each other and has to handle many requests, tracing becomes a heavier task. Sampling may then be absolutely necessary to handle and store such quantities of data. The trade-off is typically between increased system performance and amounts of errors detected by the system. As extensive sampling techniques are put in place to reduce the overhead of distributed tracing, fewer error might be caught, thus losing critical data that could be of use when debugging enterprise software. It is also important to consider different deployment strategies, since communication between nodes can be expensive and come with increased overhead.

The following research is conducted together with Nasdaq, which requires monitoring

tools to evaluate their complex systems. Nasdaq is a company that handles stock exchange where up to millions of transactions a day have to be handled (Nasdaq 2023). It is therefore indispensable to have low performance overhead of monitoring tools to be able to handle those transactions with as low latency as possible.

1.2 Research Question

Sampling may be necessary to be able to handle larger quantities of data to reduce the performance impact on a system. There are sampling techniques worth investigating to see its effect on a cloud environment and identify potential trade-offs. The research question can be formulated as the following:

Is tail based sampling better than head based or is a mixture of those the best solution in terms of CPU usage, memory usage, network usage, and error detection in a distributed cloud environment for different deployments and number of nodes?

1.3 Method Used

A Kubernetes cluster is built using Virtual Machines (VMs) as nodes and two different setups of nodes are tested, one cluster with two nodes and a second cluster with four nodes. For each node setup, a balanced and unbalanced distribution is tested. A balanced variant has as fewer messages sent between nodes compared to an unbalanced setup that has more messages travelling between the nodes. For each balanced and unbalanced setup, head based, tail based and a mixture of head and tail based sampling is tested, with a specific sampling policy set for each method. Each setup is executed seven times and the results of the metrics measured are presented in box plots where each data point represents a run.

1.4 Limitations

This research is limited by the amount of computational resources available, both in terms of CPU cores, speed, and amount of memory. The experimental runs are not as intensive as it would be for enterprise systems running on high load, thus the impact of distributed tracing could vary for large scale systems. The tests are also conducted on a smaller application, which may not be representative for large scale applications, as more connected services have more traces that have to be tracked.

2 Related Work

This chapter introduces related work where different sampling strategies, OpenTelemetry and other monitoring frameworks have been studied. This is then put into context of this thesis project, where similarities and differences are explained. Although OpenTelemetry is in constant change, it is still important to analyse previous work to have a deeper insight into distributed tracing.

L. Zhang et al. (2023) analysed head and tail based sampling and developed *Hindsight*, a sampling technique which collects telemetry data for distributed tracing retroactively. Hindsight focuses on symptomatic edge cases, where if a problem would occur locally in a service, a trigger is sent, which then collects a sample of that trace. Only when a local event is triggered for a given trace, backtracking begins to report to every service to send its span to a collector. The goal is to reduce traffic that is sent in the distributed network, to reduce overhead. Hindsight was evaluated on three distributed systems and measured its capabilities of capturing errors. The result concluded that head based sampling handled higher throughput with lower latency, followed by Hindsight and at last tail based sampling. When capturing errors, Hindsight was able to capture all errors when there were few exceptions but started to struggle when the amount of exceptions exceeded the bandwidth of the collector. L. Zhang et al. (ibid.) research differs from this research by exploring head and tail based sampling in different environments, to identify how the behaviour might change. This is done by varying the amount of nodes in the system and also taking deployment of services in a cloud system into consideration. This research also combines head and tail based sampling to see what potential trade-offs might be and how the overhead might change.

Ertl (2021) has explored how to sample traces that can save more informative data while being as responsive as head based sampling. The partial trace sampling strategy works by setting constrains for a specific span that gets generated from a service. This allows spans that have errors or other factors of interest to be sampled more frequently. Compared to tail based sampling, all spans for a given trace are not necessary to make a sampling decision. The downside is that spans that get partially sampled can lead to incomplete traces. Ertls research is different from this research since it does not look into how OpenTelemetry put into a cloud environment behaves with different sampling strategies such as head and tail based sampling. It does not further explore the dynamic between how collectors and services are deployed and how that might affect potential overheads of a given sampling method.

Reichelt, Kühne, and Hasselbring (2021) did an overhead analysis of OpenTelemetry compared to inspectIT and Kieker, which are also monitoring tools. MooBench was used as a benchmark to measure the overhead of different monitoring tools. The experiments were performed on Raspberry Pi 4s and on an i7-4770 CPU desktop system. The result showed that Kieker performed better compared to the other frameworks when logging to a file system, although it was slower for text logging. For external data processing, Kieker with TCP performed slightly better compared to OpenTelemetry using Zipkin for sending telemetry data. OpenTelemetry and inspectIT also had more overhead compared to Kieker when increasing the call tree depth of the software. Reichelt, Kühne, and Hasselbring (ibid.) differs from this work since it is not using any sampling strategies to investigate how the overhead changes. This thesis also uses OTLP for communication instead of Zipkin in a Kubernetes environment, and try different deployments and setups of nodes.

Picoreti et al. (2018) explored how a multilevel observability could be used for cloud orchestration, with a low impact on the system's performance. By multilevel observability, the authors point to using data both from infrastructure and application level for cloud orchestration. A computer vision application was emulated and Kubernetes was used as the cloud environment. A messaging middle-ware using a publish-subscribe pattern was used to gather data from entities of the system, and Prometheus was used for metric collection and integrated in the system for horizontal pod autoscaler. The results showed that using metrics from the application level gave the overall best result. The authors recommended tracing tools as a future work for a better performance effective orchestration. This thesis does not measure the overhead of metric collection, but rather distributed tracing. This thesis also emulates systems and uses Kubernetes as the cloud environment, but focuses on how tracing with sampling may affect the overall performance overhead of the system.

Yu et al. (2016) measured the overhead CloudSeer, a monitoring tool that uses logs for anomaly detection in a cloud environment. CloudSeer works by building automata from previous logs to analyse incoming logs for error detection in the system. CloudSeer accounts for logs that have interleaved sequence, tries to detect anomalies with and without error messages, and does not require all logs to be gathered before making a decision, to make it more responsive. OpenStack was deployed in a cluster of five nodes and Elasticsearch was used as the log database. The workload simulated multiple users sending requests to OpenStack through the terminal. The result showed that it had the

lowest accuracy of 92.08%, had a satisfactory throughput, precision of 83.08% and a recall of 90%. This thesis does not use logs for error detection, but rather distributed tracing with OpenTelemetry and sampling methods to determine how many errors were sampled. The experiments are also tested in a two and four node setup, to see how the dynamic might change when the amount of nodes is increased.

3 Theoretical Background

This chapter covers key concepts of a cloud infrastructure, distributed tracing and sampling techniques which are needed to understand core concepts of this research.

3.1 Microservices

As projects grow and more developers get involved, a great challenge projects have to face is maintainability. There are many architectural design principles that one could use to face these challenges at scale, one of those being microservice architectural design. The key concepts of a microservice architecture are that each domain of the system needs to be autonomous from other domains. It also has to focus on a specific problem within the system, following the "*Single Responsibility Principle*" (Newman 2021, p.2). The microservice architecture could as a benefit make the entire system more modular and thus more maintainable in the long run, since it would easier streamline the work process across teams and domains. Figure 1 shows an example of a microservice architectural design, where an API gateway sends each user request from the user interface to the appropriate microservice. As seen in the figure, each microservice is loosely coupled from each other.

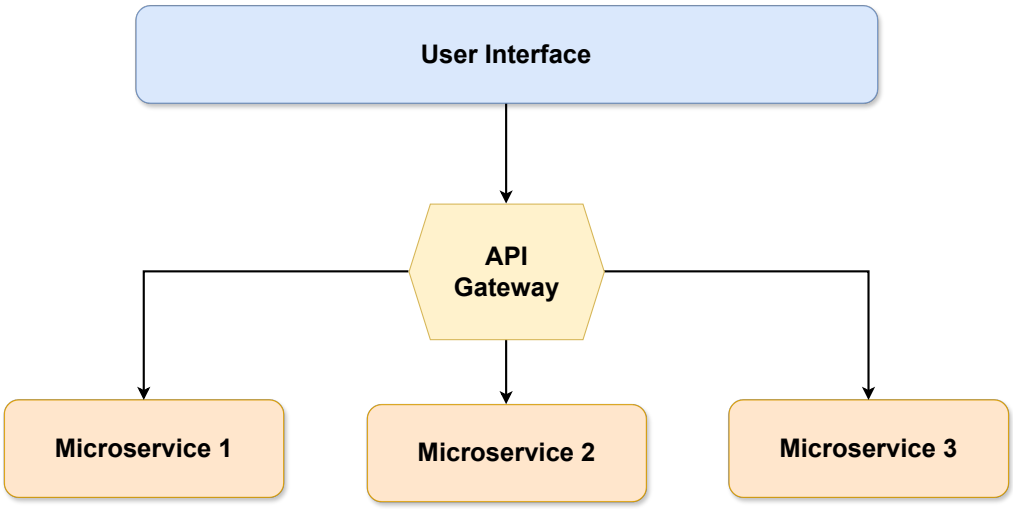

Figure 1: *An example of a microservice architectural design where an API gateway sends requests to the appropriate microservice.*

One benefit that comes with microservices as design is heterogeneity across different domains (ibid., p.4). Since each domain/service in a system has to face different challenges, different tools might be more fitted to solve those problems. Since the domains are separated by design, it becomes easier to integrate necessary tools for each

specific domain. This could in turn increase the overall performance of a system since more specialised tools would be in use rather than one-size-fits-all solutions. Another benefit that comes with the design is the resilience that the system gets from its modular design, where each service handles its own system failures accordingly (Newman 2021, p.5). Since each domain of the system is separated and autonomous, it is easier to isolate a potential breakdown of the system within a microservice. Although, one has to note that other challenges emerge from this design, such as network failures between microservices. The third benefit worth to mention is the system's scaling capability (ibid., p.5-6). Since each domain is broken up to different microservices, one can allocate more resources to only the service that needs it. Would the system be monolithic, the entire application would have to be up-scaled. Microservices can scale their system vertically (more cores/powerful machine) and horizontally (across different machines).

Throughout the years, the microservice architectural design has become more popular across companies. Thönes (2015) writes in an article how the Netflix architect Adrian Cockroft sees the design as a direct solution to handle technical debt, to have continuous delivery and to be able to scale at will. The strengths of the microservice architecture go well with cloud applications.

3.2 Distributed Systems

There are slight variations on how distributed systems are defined, but one insightful example that is similar to most definitions is from Van Steen and Tanenbaum (2017, p.2), which is as follows:

"A distributed system is a collection of autonomous computing elements that appears to its users as a single coherent system".

Computing elements could be computers, VMs or a process and are often refereed to as nodes. Each node in a distributed system can operate independently of one another, and the nodes usually communicate with each other through the network. A client that is interacting with one of the nodes in the distributed system has typically no idea that the node in turn is communicating with other nodes in the system, which may even be located in other countries. The user experience is that the entire system acts as one. To give the user this kind of experience, flawless collaboration between nodes in the

system becomes vital. The collaboration between nodes happens mainly through message passing, which makes interprocess communication integral in distributed systems (Van Steen and Tanenbaum 2017, p.163). Some common communication mechanisms are Remote Procedure Calls (RPC), REST APIs and Message-Oriented-Middleware (MOM). Since nodes can be physically placed in different parts of the world, the only possible way for a node to know the state of the system is through message passing. It is important to note that one has to account for the transit of the message, making it difficult to estimate a global state of the system. This adds another level of complexity since latency, bandwidth utilisation and packet drops are some of the additional factors that have to be considered when developing distributed systems. It is also only through the messages that developers and maintainers of the system can have a clearer picture of the current state, since they may not be able to physically access the computers. This becomes a question of observability, which in distributed systems can be explained as the ability to determine the internal state from the output of the system (Niedermaier et al. 2019). According to Sridharan (2018), observability is built upon three pillars; logs, metrics, and traces.

Logs can be explained as immutable records, which are often presented with a timestamp that explains each event that has taken place in a system (ibid., p.17). Logs can be crucial to be able to explain how an error has occurred, and often multiple events can be the cause of an error in a system. Logs are typically simple to generate, since they remind one much of print-statements, and there are tools which makes it easier to instrument. A downside to logging is that it comes with performance overhead, especially if logging occurs synchronous (ibid., p.19). There may also be additional processing required to sample the collected data, which adds more overhead to the system as a whole.

Metrics are numerical data points that are measured over time (ibid., p.21). They can be used to explain the resource usage of a system, such as the amount of consumed memory of a node or the current power consumption of a system. Metrics can be used to build dashboards that show historical trends of a system, and there are tools to ease this process, such as Prometheus. A drawback of metric collection is that it is system scoped, which makes it difficult to give a clear picture of the entire distributed system (ibid., p.23). If a request travels across multiple systems, metrics might not be sufficient to understand the entire lifespan of the request.

Tracing is the final pillar of observability. A trace is a representation of a distributed

event that has happened between different systems (Sridharan 2018, p.24). A trace shows how a request has traversed throughout the system and might give insight on how different parts of the system are connected. Traces can additionally keep track of spans for each individual request that is traversing in the system. This may be useful to identify potential bottlenecks. Tracing in a distributed system is explained more in depth in Section 3.2.1. One challenge with distributed tracing is that every component that sends and receives requests needs to be configured, so that tracing information is sent to an appropriate collector (ibid., p.27). This could be difficult to implement, and some companies may not want to spend additional resources for this feature.

3.2.1 Distributed Tracing

As distributed systems scale up and dependencies become unpredictable, monitoring tools are of great importance to maintain such systems. Distributed tracing helps developers to identify dependencies, discover bottlenecks and find the root cause of failures of a system. By using distributed tracing, one can generate directed acyclic graphs that can present causal order of different requests in the system, unlike tree-based representations, distributed tracing gives a temporal perspective (Bento et al. 2021).

To be able to generate a trace, an initial request needs to be sent to a node in the system. A trace identifier should then be generated, which is supposed to represent the entire trace (ibid.). Every node that will later on be within the trace has to report its span with the trace identifier. A span identifier should be generated as well for each request in the trace, to be able to distinguish requests from each other. Additionally, a span has a start time, its duration, and could also include an operation name for the span itself (ibid.). It is also possible to add arbitrary key-value pairs to the span for additional information that may be of interest. An example of a span could be an RPC call from one node to another. At last, a parent span identifier is needed to be able to represent the relationship between requests. If *Node A* sent a request to *Node B*, then *Span A* would be the parent of *Span B* and *Span B* would be the child of *Span A*.

Figure 2 shows an example of a trace. An initial message is sent to *Node A*, which in turn generates a trace identifier and a span identifier called *Span A*. It should then also keep track of the start time of when it received a message. It later sends a request to *Node B*, with the trace identifier, and its span identifier so that a parent/child

relationship may be created. *Node B* sends requests to *Node C* and later on *Node D*. During the lifetime of *Node D*, it in turn sends a request to *Node E*. At last, *Node A* sends a request to *Node F* and the tracing for this instance stops.

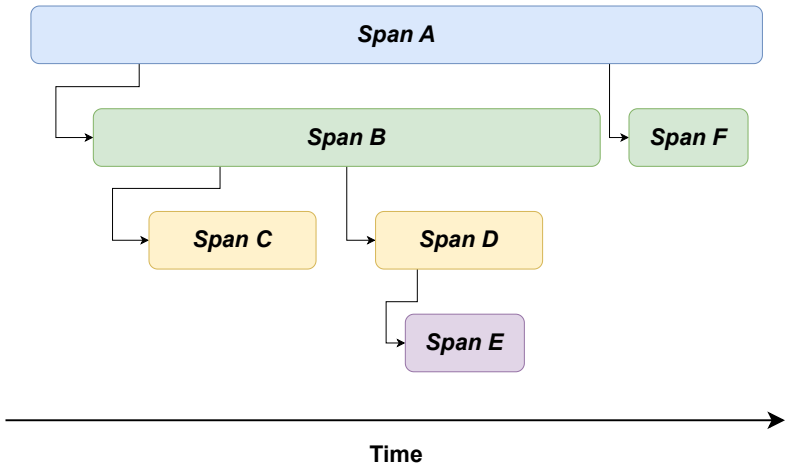

Figure 2: *A trace where Node A makes a request to Node B, which later makes a request to nodes C and D. Node D makes a request to Node E and at last, Node A makes a request to Node F. A span is generated for each request sent and the width of each box represents the span time.*

During the generation of a trace, each span sends its record to a collector or logs it, and they are usually logged asynchronously to the disk (Sridharan 2018, p.25). The challenges with tracing are that it can be difficult to integrate to the codebase. Every component that either sends or receives messages from nodes needs to be modified to be able to trace the messages. Another challenge is that programs that are built using open source frameworks might require constant modifications as they change to enable distributed tracing (ibid., p.27).

3.3 Containers

In a distributed system where one has deployed microservices, different services might require different kinds of resources to operate as expected. Some of these resources might be packages, libraries and even Operating Systems (OS's), all while having to operate on the same computer (node). One solution to this problem is virtualisation, as it enables isolation and virtualisation of resources in a computer.

VMs are a way of emulating an OS and abstracting away the hardware level (T. Siddiqui, S. A. Siddiqui, and Khan 2019). The layer that makes the virtualisation of hardware resources come true is the *hypervisor*. Not only is the hypervisor managing

virtual resources, but it also has to make sure that the host OS can execute the commands given by the VMs, since the VMs run in a non-privileged mode (Q. Zhang et al. 2018). The VMs run through a hypervisor, which hosts one or multiple VMs on top of the host OS. It is a way of dividing up the hardware resources across different OS's using virtual resources. A container is another way to virtualise an environment, and compared to VMs, they consume less resources and time (Pahl et al. 2017). Containers do not have a hypervisor since they do not have to emulate an entire OS, they instead have a container engine. A container engine is responsible for sending system calls from the container to the host OS. A container contains the application executable, a bundle of libraries and other dependencies to be able to operate successfully (Q. Zhang et al. 2018). It does not need to create an OS for each container that is running. A container can essentially be seen as a process running on a host OS with restrictions. Figure 3 shows the main difference between a VM and a container. As seen in the figure, the container does not need an OS in its stack for each container, but rather each container shares the same host OS through the container engine.

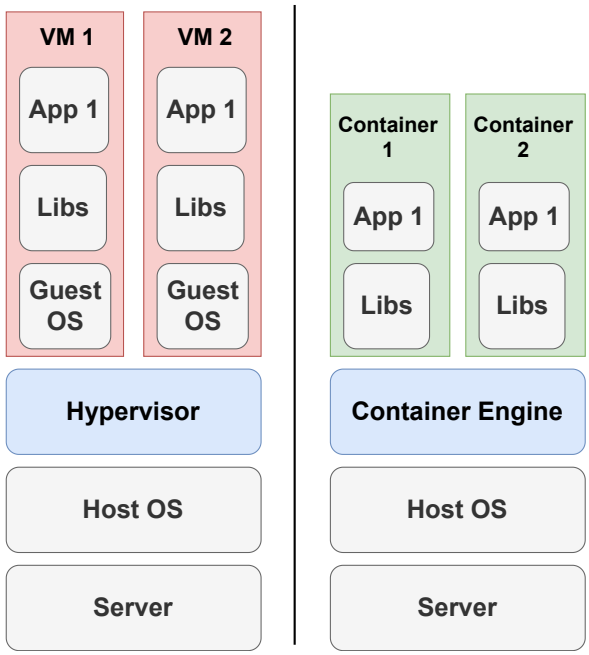

Figure 3: *Difference between a VM and a container architecture. Two VMs running on top of a hypervisor and two containers running on top of a container engine.*

Containers are separated from each other through namespaces and control groups (cgroups) (ibid.). Cgroups are a way of controlling the resources of a system, such as memory, CPU, and network. These resource allocations can change dynamically during the containers' lifetime, but are restricted to the limit specified in the cgroup.

Namespaces add another level of abstraction to the resources, which might be host names, process IDs and network interfaces. If two containers have a process running with the process ID 1234, they would not conflict since they would be on different namespaces. The containers can communicate with each other and to the public internet by mapping a network interface from the container to the host OS or another container. One has to make sure that there are no conflicts if the network interfaces from two containers are mapped to the host OS.

Because of the lightweight nature of containers compared to VMs, they can easily be deployed and shut down on demand. This can for instance make it very fitting for cloud native applications following a microservice architecture. Containers might be a practical and straight forward solution if a microservice has to be deployed to quickly handle a workload, and after it has finished, be torn down. It is also possible to use VMs and containers combined, as VMs can host container instances. This can in turn give a great level of isolation and scalability.

3.4 Kubernetes

Kubernetes, also known as K8s, is a container orchestration system used for scaling, management and deployment of containerised applications (Red Hat 2020). It was originally created by Google in the Borg project, but was later open sourced and donated to the Cloud Native Computing Foundation (CNCF) in 2015. It can be used to control clusters deployed in the cloud, data centers or even on a consumer computer. Kubernetes can run on bare metal (directly on hardware) as well as on VMs, making it versatile for a wide range of deployments.

There are two types of machines that are deployed in a Kubernetes cluster, each having their own responsibility. The first type is a worker machine, which is a called a node or a worker node, and every cluster has at least one worker node in a Kubernetes cluster (The Kubernetes Authors 2024). The worker node is the one that handles workloads that get deployed in the cluster. A node can have one or more pods, which is the smallest Kubernetes object (The Kubernetes Authors 2021). Each pod can have one or more containers deployed inside of it and have sidecar containers that can add additional features to the main application, such as observability. Every worker node is then controlled by one or more master nodes called control plane, that manages the cluster. The control plane is responsible for exposing the API of the cluster and different interfaces for deployment and management of pods in the cluster.

A control plane consists of five main components that make sure that the cluster can run efficiently, have fault tolerance and high availability. The first component is *etcd*, which is a distributed key-value data store mainly used for storing configurations and the current state of the cluster (The Kubernetes Authors 2024). The second component is the *kube-apiserver*, which exposes the Kubernetes API server. The API server makes external and internal communication possible through interfaces. *Kube-scheduler* is another component in the control plane, which handles the scheduling of new pods. When a new pod is created, it has to be assigned to a node and the kube-scheduler decides on which one, depending on factors such as hardware, software, or policy constraints, data locality, deadlines and much more (ibid.). The fourth component is the *kube-controller-manager*, which runs controller processes. It communicates with the API server to handle resources such that the cluster can be in an optimal state. At last, there is the *cloud-controller-manager*, which communicates with a cloud provider and allows coordination between the cloud and the cluster (ibid.).

A worker node consists of three components, where the first one being the *kubelet*. The kubelet is responsible for making sure that containers are running inside a pod correctly (ibid.). Each worker node in the cluster has a kubelet. The second component is the *kube-proxy*, which assures that network rules are maintained on each worker node. Network rules give pods the ability to communicate internally and externally in the cluster. The last component that a worker node has is the *container runtime*. The container runtime makes sure that the life-cycle and execution of a container is healthy. The kubelet can communicate with the container runtime through the Container Runtime Interface (CRI) (The Kubernetes Authors 2023).

A cluster will vary for each system according to the desires and goals of specific applications. Figure 4 shows how such a Kubernetes setup can look like. The master node is in control of the control plane, which consists of four components. Note that a single master node does not have to control all the four components, they can be split up across multiple nodes. Each worker node in the figure has its three main components and pods, which can be one or more. Each pod in turn has one or more containers which carries the workload.

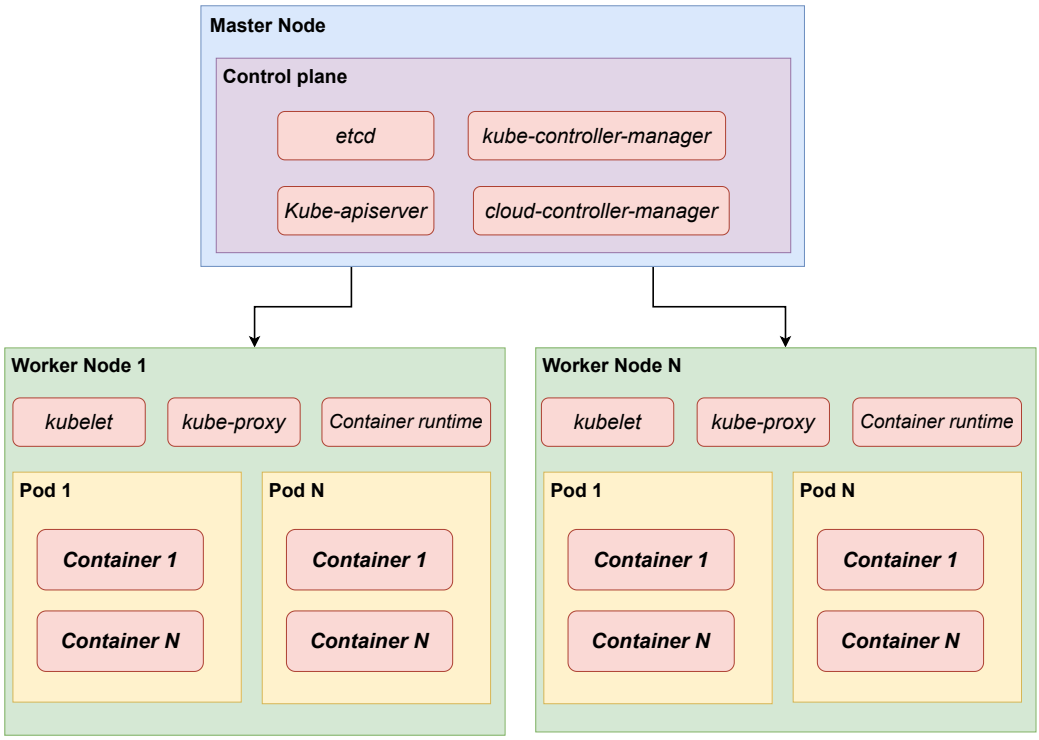

Figure 4: *An example of a Kubernetes cluster setup, where a control plane manages one or more worker nodes. Each worker node has one or more pods, which in turn contain one or more containers.*

3.5 OpenTelemetry

OpenTelemetry comes from the merge of two prior projects, OpenCensus and OpenTracing (OpenTelemetry Authors 2024e). The project belongs to the Cloud Native Computing Foundation (CNCF) and is an observability framework that has tool-kits for logging, metrics and tracing. It supports a broad range of programming languages and goes well with other observability back-ends such as Prometheus, commonly used to monitor cloud systems and Jaeger, a distributed tracing platform. The goal of OpenTelemetry is to provide a standardised way of handling telemetry data, that observability back-ends could later process.

OpenTelemetry has several key components that make it a versatile tool and one of them is its specification. The specification allows OpenTelemetry to work on cross-language platforms and defines the API, SDK and data which consists of the OpenTelemetry Protocol (OTLP) (OpenTelemetry Authors 2024b).

The *collector* is one of the key components of OpenTelemetry and is the component that logs, metrics, and traces are sent to from services in a cluster. The collector

consists of four pipelines which are receivers, processors, exporters, and connectors (OpenTelemetry Authors 2024c). The receiver receives data from one or more data sources in a system, and each collector needs at least one receiver to be able to operate. The collector can be easily configured to meet the specific needs of a system. It is possible to define what types of telemetry that will be collected, where it will be collected from (e.g. Kafka) and which protocol that will be used for the collection. The receiver can either be pull or push based, depending on the needs of the system. In the second step of the collector pipeline, there is the processor(s). It allows the collector to transform or modify all the data that is sent from the receiver, and one can add zero or more processors in the pipeline to customise a desired output. Some jobs that a processor can perform are recalculating incoming telemetry data, renaming data or sampling it. Sampling will be explained more in depth in Section 3.6. Some specific processor pipelines are memory limiter, sampling and batching, but one can add a custom processor in the chain if needed. The third process in the collector pipeline that comes after the processor is the exporter. The exporter sends data to one or more endpoints and can, as the receiver, also be configured to act pull or push based. It can export data using HTTP or gRPC, depending on how it is configured. A collector requires one or more exporters to be able to function. In the exporter, one can also configure security related settings, such as setting up tokens or enabling TLS. The last step in the collector pipeline is the connector, which connects two pipelines, giving it the ability to act both as a receiver and exporter. The connector is useful for replicating, routing or emitting data. There are two sorts of collectors, stateful or stateless. A stateful collector requires context received within a time-span to know how to handle telemetry data. A stateless collector can handle messages as they are received, without any context from previous data.

Another component in the OpenTelemetry toolkit is the *instrumentation* mechanism (ibid.). By applying instrumentation to services in the cluster, the services can send telemetry data to the collector correctly. There are two ways of applying instrumentation to an application, where the first one is being code-based. The code-based approach requires developers to import OpenTelemetry libraries to manually implement where traces and spans should be recorded. It is possible to modify the SDK and API to make best use of the available tools for the specific programming language (OpenTelemetry Authors 2024a). The data needs to later be exported either from the service itself or through a collector. The second alternative for instrumentation is by using a zero-code solution. The zero-code solution lets one use instrumentation

without having to manually implement any additional code to services. It works by adding an exporting mechanism to a program, which is installed as an agent to the program (OpenTelemetry Authors 2024f). Depending on the programming language, the zero-code solution utilizes byte-code manipulation, eBPF (running programs in privileged context) for call injection or monkey patching (changing the behaviour of a program during runtime).

The last component that the OpenTelemetry toolkit provides is the *Kubernetes Operator*. The K8s operator can manage both the collector and the auto-instrumentation, which is used as a zero-code solution for instrumentation (OpenTelemetry Authors 2024d). The operator then makes sure that the auto-instrumentation is injected to the new program running inside a pod, when it is created in the cluster.

Figure 5 shows an example of an OpenTelemetry deployment in a Kubernetes cluster. The Kubernetes Operator notices that there is a new pod and makes sure that an auto-instrumentation is loaded next to it, if it has the correct annotation. The operator injects into the program so that it can emit telemetry data. As the program functions, it emits telemetry data to the collector with a specified protocol. The receiver inside the collector takes the data and later sends it to the processor. The processor has a memory limiter in its first pipeline and later a sampler. When the processing is completed, it is sent to the exporter, which sends the data to a Prometheus and Jaeger back-end.

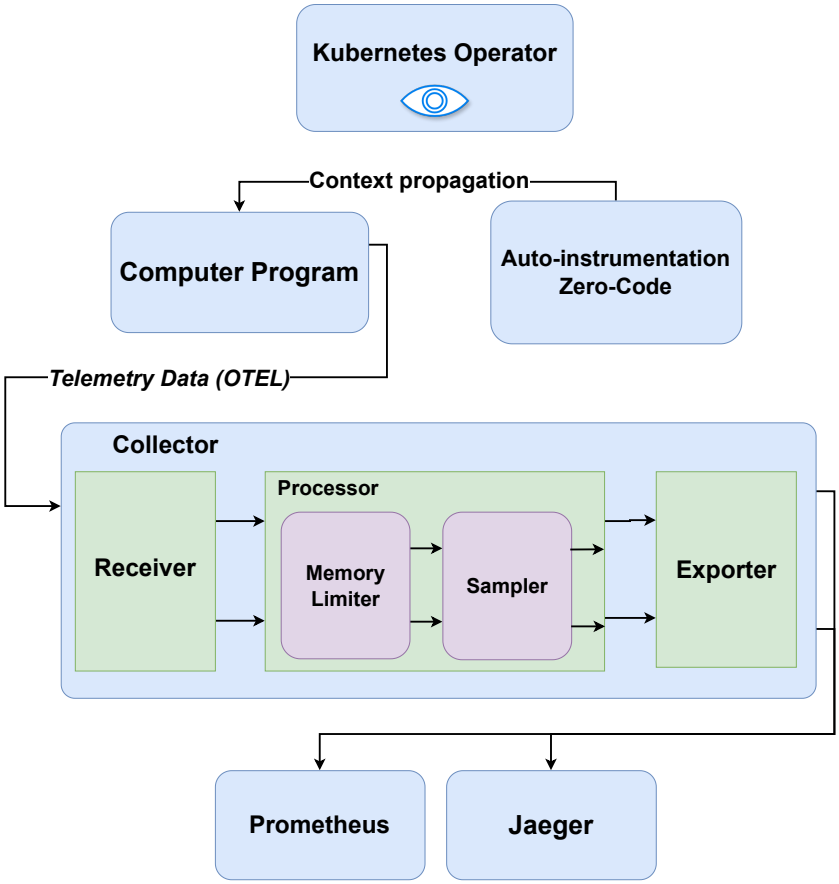

Figure 5: *A deployment of a collector using auto-instrumentation in a Kubernetes cluster where a trace is sent from a computer program to a collector, which is later exported to two back-ends.*

3.6 Sampling Methods

As systems scale up and become more complex, more telemetry data will be generated and stored in the system. The amount of telemetry data that has to go through the system and later on be stored comes at a cost. The cost will vary on different factors, such as CPU, memory, network and disk usage, thus sophisticated methods have to be in place to handle the additional overhead telemetry collection might bring to a system. One way of reducing the quantity of data that goes through the system is through sampling. By implementing sampling, one can decide if telemetry data should be stored. OpenTelemetry has currently two ways of implementing sampling, head and tail based sampling (explained in depth in Sections 3.6.1 and 3.6.2) (OpenTelemetry Authors 2023). The sampling step occurs in the OpenTelemetry collector processor, and different parameters can be configured for each sampling method to reach an

optimal point.

3.6.1 Head Based Sampling

The head based sampling strategy tries to make an early decision if it should proceed with sampling. This is possible by not making a sampling decision on the content of the trace and its span. Head based sampling works by setting a percentage on how much one wants to let through in the collector. A hash seed can be set to get a deterministic output out of the head based sampler. The sampler decides if it should sample a trace by hashing the trace ID together with the hash seed. It is important to hash the trace ID on all spans so that all spans for a certain trace ID get collected or dropped. Otherwise, the head based sampling could have gaps in its traces. Depending on the percentage set, more or less traces will then be sent through the collector. A sampling percentage of 100 samples all the traces.

The upside of using the head based sampler is that it is easily configurable, and the sampling process can be implemented at any point in the trace collection pipeline (OpenTelemetry Authors 2023). Another upside is that the head based sampler does not have to temporarily store any data to make any decisions. This reduces the amount of memory that is required to make the sampling decision. The downside of using the head based sampling strategy is that no sophisticated decisions can be made when making a sampling decision. This could for instance be of importance if one wants to catch rare errors that might occur in a system. Another downside is that if it is important to identify rare bottlenecks that have high latency, a head based sampling decision can filter those rare cases. That is because head based sampling samples based on a probabilistic value, if a rare request would be sent that has high latency, the head based sampling method could miss to sample that trace.

Figure 6 shows an example of a collector that has a head based sampling process, where the sampling rate is set to 50 percent. Two spans with different trade IDs are sent to the collector, and only the span with the trace ID 1 is sampled, after the trace ID has been hashed. All spans with trace ID 1 will always be sampled as they output the same value after the trace ID is hashed, and therefore spans with trace ID 2 will always be dropped.

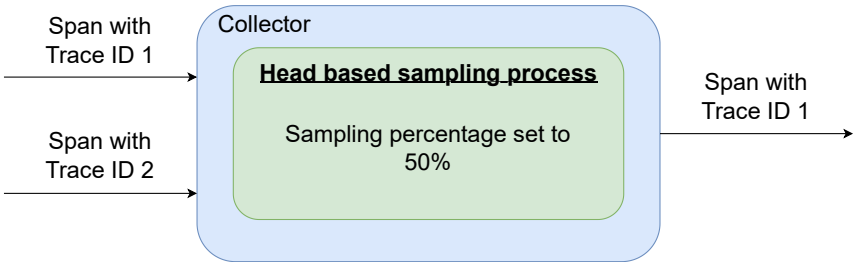

Figure 6: *A collector with a head based sampling process, where the sampling rate is set to 50 percent. The span with trace ID 1 gets sampled and the span with trace ID 2 gets dropped.*

3.6.2 Tail Based Sampling

The tail based sampling strategy waits until all or most of the spans have been collected before making a sampling decision (OpenTelemetry Authors 2023). The spans are sent to the collector and are stored there until a sampling decision can be made. Since the entire trace is collected, better decisions can be made if the trace is worth sampling. It is possible with tail based sampling to always pass through messages with certain messages/attributes, such as errors, warnings, or debugs. This can be useful to catch rare occurring errors that head based sampling could have missed. It is also possible to always sample messages from a specific microservice in a distributed system, which is useful if a newly integrated service has to be constantly monitored, to avoid unexpected behaviours. Since the entire trace is recorded, one can sample based on the latency of the trace or span, which is useful to identify bottlenecks in a large and complex system. To avoid oversampling when services let through spans that might be unbearable for the collector, one can set a sampling rate on traces and these can also be modified based on specific conditions. It is also possible to set how many traces that are expected per second to help the system allocate appropriate amount of memory for data structures before collecting the traces. Figure 7 shows an example of a tailed based sampling process. Two spans with different trace IDs are sent to the collector, where the span with the trace ID 1 has 200 milliseconds (ms) latency, and the span with trace ID 2 has a latency of 20 ms. Only spans with trace ID 1 gets sampled, as one of its spans has a latency above 100 ms.

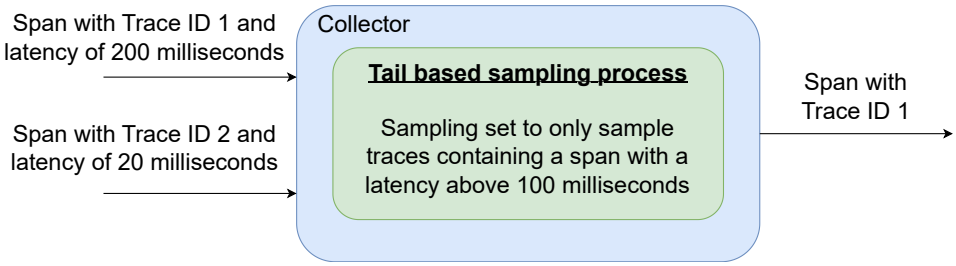

Figure 7: *A collector with a tail based sampling process, where only traces containing a span with a latency above 100 milliseconds gets sampled. Span with trace ID 1 gets sampled and span with trace ID 2 gets dropped.*

Since services can be requested asynchronously, it is not always possible to know when all spans have been collected or if there is still a request pending in the background. To solve this issue, the collector holds the spans for a set time before making a decision (OpenTelemetry Github Contributors 2024). The current solution in place does not guarantee that all spans will be collected before making a sampling decision if a span is not within the time limit of the decision wait parameter. Therefore, a better understanding of the system behaviour is needed to make an optimal decision. One aspect that has to be considered when raising the decision wait parameter is that more traces and spans will be stored in memory before sending it out from the collector and to the back-end.

Another challenge with tail based sampling is that sampling policies may have to change as the system changes. Since one can sample on system specific messages with additional conditions strongly related to a service, one has to constantly maintain the policies. If systems get either removed or added, modifications to the policies might be needed.

If a collector with a head based sampling process is under heavy pressure, one can scale up the amount of collectors to handle the increased workload. This can put less pressure on a single collector, removing potential bottlenecks of collecting telemetry data in a system that scales. Since the collector holds spans from traces and later decides if the message should be sampled, the collector has to be stateful. This means that when services send their spans from a specific trace, they all have to be sent to the same collector, or fragmentations will appear in the trace and incorrect decisions could be made in the head sampling process. When having a stateful collector, a loadbalancer is required between the sampling collectors and services that send telemetry data to those collectors, if there are many collectors in the system. The loadbalancers job is to always

send telemetry data from a certain trace to the same collector to avoid fragmentations. This can be achieved by having the loadbalancer hashing the trace ID and from the result deciding on which collector to send the trace to. Note that the loadbalancer needs to know in advance how many collectors with a sampler are deployed in the cluster. Figure 8 shows how multiple services can through the loadbalancer reach the right collector. Each trace ID gets hashed in the loadbalancer and later through the output of the hash routed to the appropriate collector.

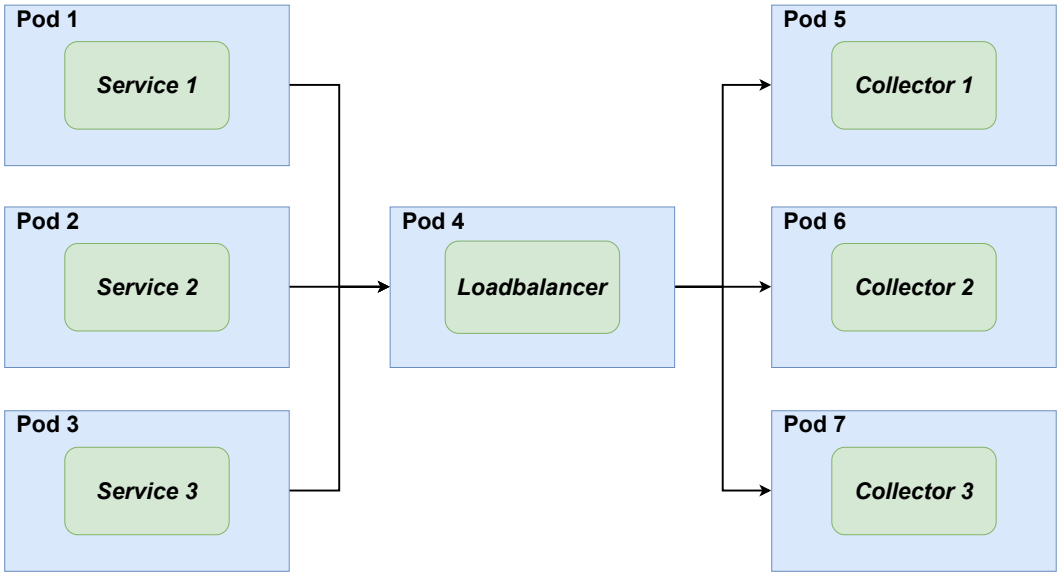

Figure 8: *A deployment of a loadbalancer with multiple collectors deployed in a cluster.*

4 Method

This chapter explains how the services are deployed in the Kubernetes cluster and how they communicate with each other for the test cases. Collector placements are also explained for the different sampling strategies.

4.1 Program Structure

Nasdaq has a distributed system with services communicating with each other whenever a request is sent through a gateway. The service is provided and deployed for clearing houses and tries to match a buyer with a seller. A transaction has to be verified, there might be risk calculations involved, and service level agreements have to be fulfilled before a transaction is made. The service deployment might look different for different customers depending on their needs.

The system's services, how the services are connected, and the data sent in-between have been analysed, and an emulation was created for this experiment, where a call chain of services are deployed in a Kubernetes cluster. Each service is connected with zero or more services, where it is zero if it is a leaf in the call chain. Each service that is connected with another service can send one or more messages and receives a response later on from that service. The initial request is sent through an API gateway, and the total amount of spans from a single request is 111 spans. Figure 9 shows how the call chain program is connected. An initial request is sent from the user to the API gateway, and from there the API-Gateway sends requests to other services in the system. The service *"Key-Cloak"* for instance will receive 18 requests from a single request initially sent from the user. All the messages sent in-between add up to 56 messages, including the initial one sent from the user. When a message is sent, a span is generated, but it is also generated from a service that receives a message. This makes the total amount of spans 56 times two, which is 112, but the user never responds, which makes it 111 spans from a trace. The names of the services have been modified to not leak information about Nasdaq internal systems.

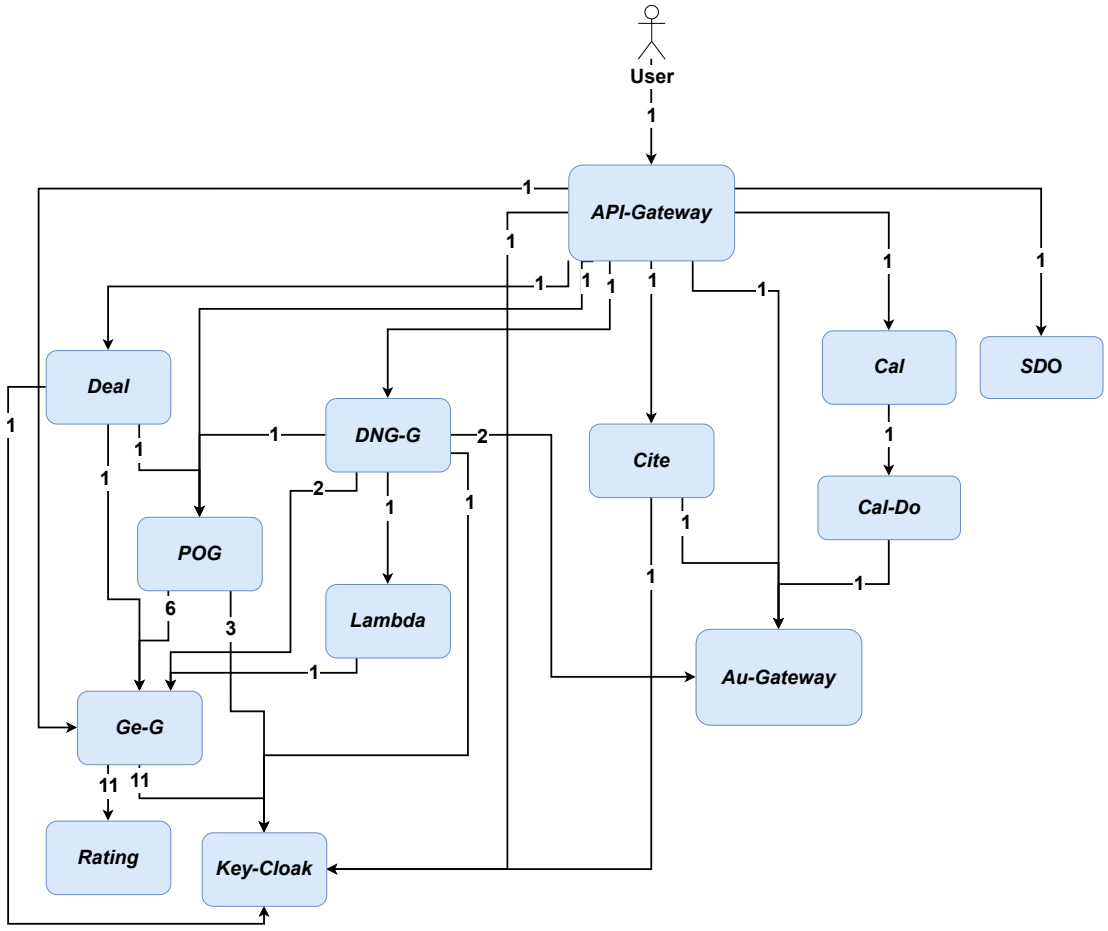

Figure 9: *How the call chain program with its services are connected and the amount of messages that are sent from each service. A request is sent from the user to the API gateway. The API gateway sends requests to services, which in turn sends requests to other services. A single request from the user generates 111 spans, where spans are sent from each service to a collector.*

4.2 Collector Deployments & Configurations

The three sampling strategies that are tested have the same collector deployments, but with different configurations. All the services of the call chain application are injected with auto-instrumentation to enable trace collection. The spans are sent with HTTP using OTLP, which is the standard OpenTelemetry communication protocol. The spans are sent to a collector that is deployed on the same node as the service itself, which is called a daemonset collector. The daemonset collectors later send their spans to a loadbalancer with gRPC using OTLP. The loadbalancer redirects the spans it receives to one of the two collectors that are deployed after it in the collector chain with gRPC using OTLP. The collector that the loadbalancer sends its spans to depends on the trace ID of the span. Figure 10 shows how the collectors communicate with

each other. The amount of daemonset collectors is the same amount as nodes in the Kubernetes cluster. The experiments are tested on two and four nodes, to see how the dynamic between services and collectors changes when the number of nodes is increased. Since two and four nodes are tested, the daemonset collectors are two or four. The final destination of spans is Jaeger, which acts as the back-end to store the spans. They are sent using gRPC with OTLP.

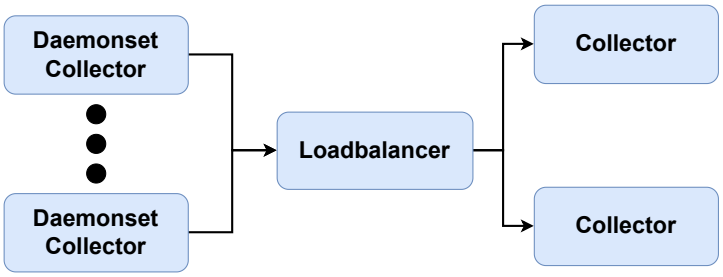

Figure 10: *How the collectors are deployed and communicate with each other, where spans are sent between the collectors and loadbalancer.*

The setup for the head based sampling process happens in the daemonset collector and the sampling percentage is set to five percent. The head sampling process is set as early as possible in the chain so that no spans are sent more than it has to in the cluster.

The setup for the tail based sampling is on the collectors after the loadbalancer. It only samples traces that contain error messages with a status code of 400 and up. A status code is given by a server in response to a request made by the user for HTTP messages. Its decision wait time is set to 10 seconds. The tail based process has to be placed after the loadbalancer so that every collector gets all the spans related to a specific trace ID.

The mixed sampling setup has a head based sampling process in its daemonset, where the sampling percentage is set to 20 percent. A tail based sampling process is set for the collectors that are after the loadbalancer. As the tail based sampling method, the mixed sampling only samples error messages with a status code of 400 and up, and has its decision wait time set to 10 seconds. The mixed sampling setup has a higher sampling percentage at the daemonset compared to the head based setup, and this is because a stricter sampling decision is made later in the collectors with tail based sampling. It is also of interest to observe the dynamic with a less strict sampling percentage at the daemonset. Figure 11 shows how the mixed sampling

setup is deployed. A span is sent to the daemonset collector, which has a head based sampling process with a sampling rate set to 20 percent. The span is later sent to a loadbalancer, which sends the span to an appropriate collector with a tail based sampling process. The tail based sampling process only samples traces that contain a span with status code of 400 and up.

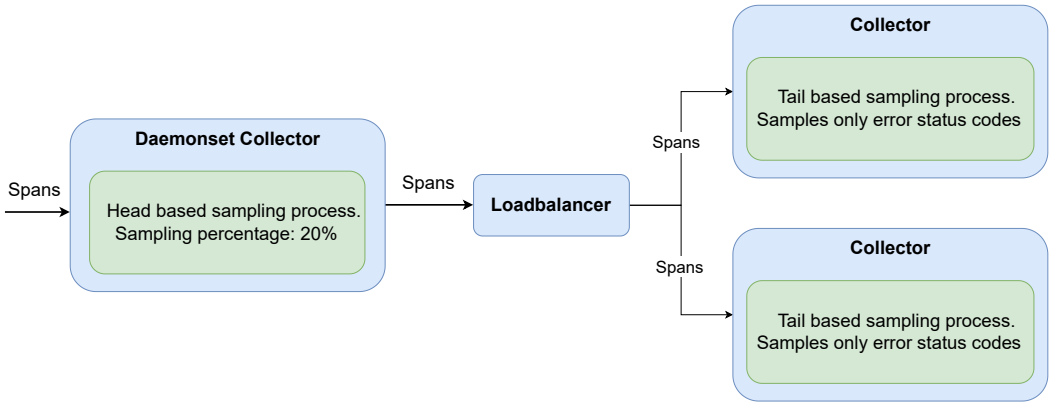

Figure 11: *The mixed sampling setup, with a head based sampling process in the daemonset and tail based sampling in the collectors after the loadbalancer. The head based sampling percentage is set to 20 percent and the tail based sampling process only samples HTTP status codes of 400 and up.*

4.3 Metric Collection

Prometheus is used to scrape metrics from the Kubernetes cluster to determine the performance of the different setups. Requests are sent to the Prometheus server that is placed on one of the nodes, for each metric that is being evaluated after the collectors have handled every trace. When sending requests to the Prometheus sever for metric collection, a data collection interval of five seconds is set.

Jaeger is used as a back-end to gather all traces sent from the collectors after the loadbalancer. When a run is finished, all collected traces from Jaeger are gathered by sending a request to it. The size of all traces is calculated and all caught error messages are summed up to calculate its error detection rate. The calculation of total time taken for a run is done by calculating the difference between the start and end time after all requests have been received by the end user.

To calculate the CPU usage of the collectors and Jaeger for a node, CPU working seconds is scraped, where idle time is excluded. By measuring the CPU working seconds, all other noise from the system is excluded, thus giving a better comparison

between the sampling strategies. The difference from the start time to the end time of the CPU work seconds for a run is used as the result. To calculate the memory usage of the collectors and Jaeger for each node, current allocated memory is gathered from Prometheus on every time interval and the values are summed up at the end. The calculation of network traffic is done by scraping received bytes on every time interval during the run for each node in the system. The difference from the start and end time is calculated to get the final result of a run.

4.4 Deployments On Nodes

This chapter describes how each service is deployed in the Kubernetes cluster for a two and four node setup. For each of the two and four node setups, different deployments of services are presented. The deployments are balanced and unbalanced, and they differentiate by having the call chain app sending a high amount or a low amount of requests between nodes in the system. The balanced deployment of services sends fewer messages between nodes compared to the unbalanced deployment.

4.4.1 Two Node Deployment

Table 1 shows the specifications of the two nodes in the cluster. All the nodes have four gigabytes of memory, two CPU cores and 3.3 gigahertz of clock speed on the CPU. The control plane is in node 1, but is also a worker.

Table 1: *Cluster setup for two VM nodes.*

Node	CPU Cores	CPU clock speed (Gigahertz)	Memory (Gigabytes)
Control Plane + Worker 1	2	3.3	4
Worker 2	2	3.3	4

For the two node setup, there is a daemonset collector on each node that sends their spans to the loadbalancer deployed on node 2. From there, the loadbalancer sends its requests to one of the two collectors deployed in node 1 and node 2. Which collector that gets which span depends on the trace ID of the span. It can be expected that the distribution is quite even.

The deployment of services is divided up by how many request they send between nodes. Figure 12 shows the deployment, where low requests are sent between the services. A total of seven requests are sent between the nodes for a single request sent from a user, as the services still have to communicate. This is a balanced setup where the goal is to send few messages between nodes. Figure 13 shows unbalanced deployment of services where the amount of requests is larger compared to the balanced setup. The unbalanced setup sends a total of 34 requests between nodes, because services have been deployed in a way that forces more communication between nodes.

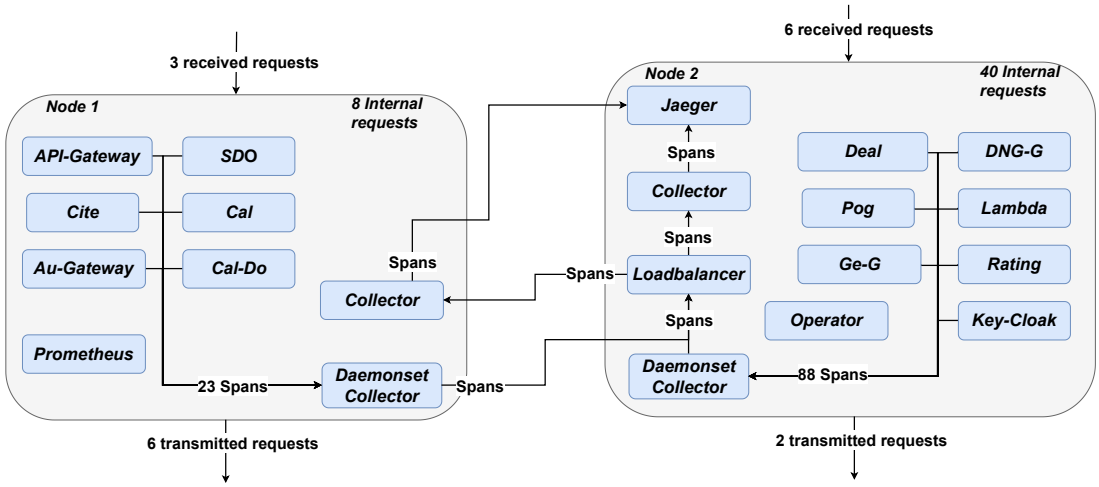

Figure 12: *Services deployed balanced to minimise requests sent between nodes, for a two node setup. The amount of requests sent are generated from a single request by a user that sends a message to the API-gateway. The services in turn send generated spans to the local daemonset collector, which then are sent to a loadbalancer. The loadbalancer sends the spans to a collector, which at last sends it to Jaeger.*

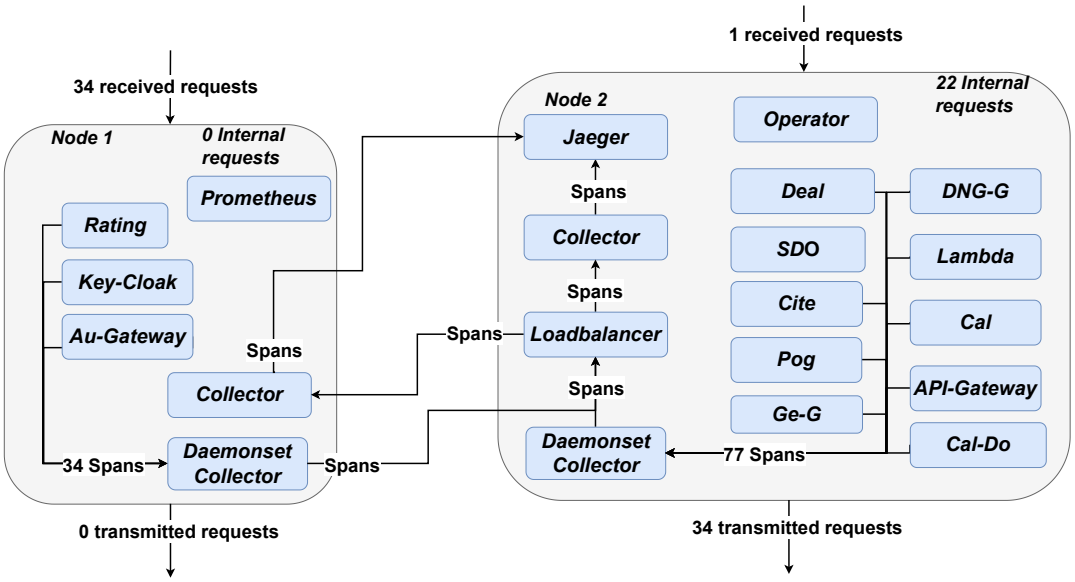

Figure 13: *Services deployed unbalanced to maximise requests sent between nodes, for a two node setup. The amount of requests sent are generated from a single request by a user that sends a message to the API-gateway. The services in turn send generated spans to the local daemonset collector, which then are sent to a loadbalancer. The loadbalancer sends the spans to a collector, which at last sends it to Jaeger.*

4.4.2 Four Node Deployment

Table 2 shows the cluster setup for four nodes. All the nodes have the same amount of CPU and memory resources. Prometheus is deployed on the control plane and the nodes in the system all have 3.3 gigahertz of clock speed with two cores each. Each node has four gigabytes of memory.

Table 2: *Cluster setup for four VM nodes.*

Node	CPU Cores	CPU clock speed (Gigahertz)	Memory (Gigabytes)
Control Plane + Worker 1	2	3.3	4
Worker 2	2	3.3	4
Worker 3	2	3.3	4
Worker 4	2	3.3	4

On the four VM setup, there are four nodes, which all have a daemonset collector each. They all later send their spans to the loadbalancer, which is placed in node 2 and the

loadbalancer later sends the spans to one of the collectors deployed in node 3 or node 4. As mentioned previously, it is upon the trace ID of each span that decides where the spans are sent to from the loadbalancer, but it can be assumed that the distribution is even.

Two different tests are performed on the four node setup, where the distribution of requests sent from each node is either more or less balanced. Figure 14 shows the balanced deployment of services where all the nodes combined sends a total of 17 requests outside its node. Figure 15 shows the deployment for the unbalanced setup, where the amount of sent requests are more compared to the balanced setup. The unbalanced setup sends a total of 50 requests outside its node.

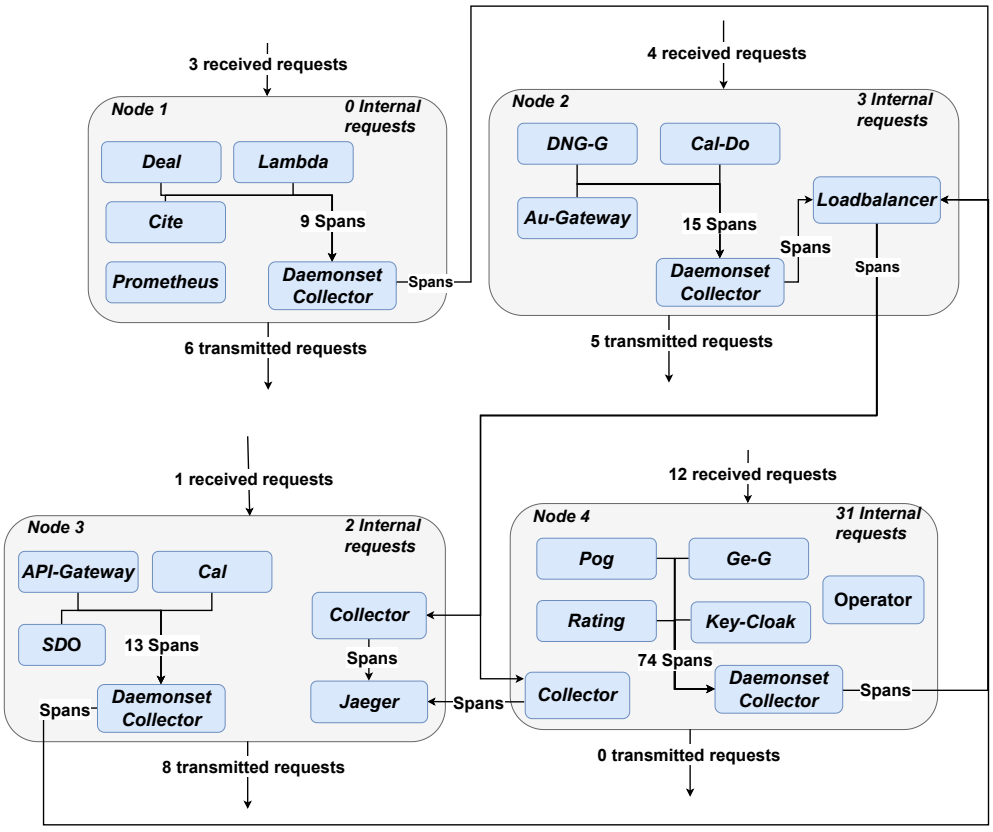

Figure 14: *Balanced deployed services to minimise requests sent for a four node setup. The amount of requests sent are generated from a single request by a user that sends a message to the API-gateway. The services in turn send generated spans to the local daemonset collector, which then are sent to a loadbalancer. The loadbalancer sends the spans to a collector, which at last sends it to Jaeger.*

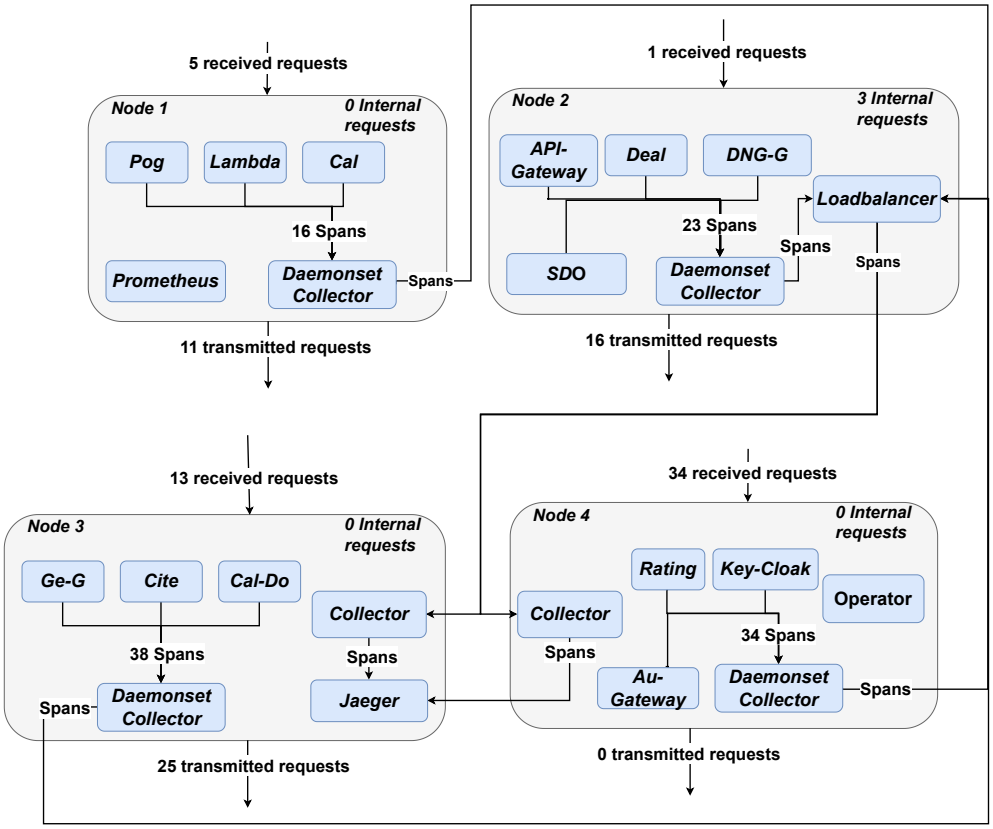

Figure 15: *Unbalanced deployed services to maximise requests sent for a four node setup. The amount of requests sent are generated from a single request by a user that sends a message to the API-gateway. The services in turn send generated spans to the local daemonset collector, which then are sent to a loadbalancer. The loadbalancer sends the spans to a collector, which at last sends it to Jaeger*

4.5 Workflow Of Experiments

For each of the three sampling strategies, an experiment is performed for a balanced and unbalanced deployment of services. Each experiment is executed seven times to reduce the impact of background noise that may occur during a run. A load generator is started, which is placed outside the cluster. It starts three processes and for every process 230 requests are sent to the cluster. Between every sent request, the process sleeps for 0.1 seconds before sending another request. Of all the sent requests, ten percent of the requests contain an error status code, which generates an error in the cluster. The error is used to see how much error head, tail and mixed sampling managed to pick up, as different sampling policies may affect its capabilities to capture traces of interest.

Between the balanced and unbalanced runs for head, tail and mixed sampling, the entire cluster is cleaned so that no memory gets built up as the experiment proceeds.

It is also ensured that every deployment is running before the load generator starts. The entire process happens for both the two and four node setup.

5 Results

This chapter present the results from every experiment for both the two and four node setup.

5.1 Two Node Setup

Figure 16 shows the time taken for sending a request and receiving a response for both balanced and unbalanced setups for each sampling strategy. The differences between the sampling strategies are slim, but a larger difference can be observed between an unbalanced and balanced setup. The unbalanced deployment for head based sampling had 6% longer runtime compared to the balanced deployment. For mixed and tail based sampling, the unbalanced deployment took 5% longer than the balanced deployment.

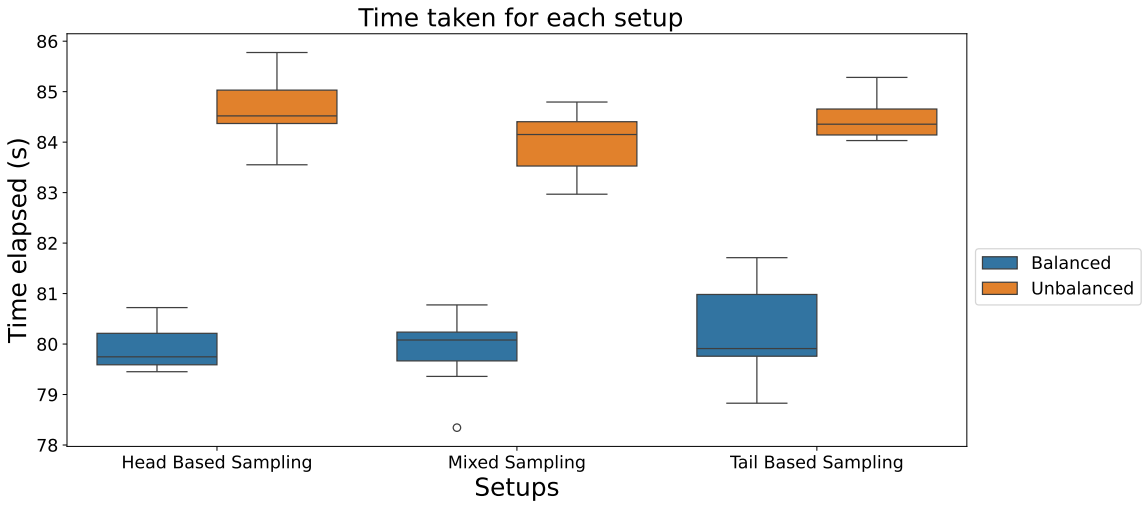

Figure 16: *Time taken for two nodes to handle all user sent requests for balanced and unbalanced setup, for each sampling strategy. The unbalanced deployment took 6% longer to execute compared to the balanced deployment for head based sampling. For mixed and tail based sampling, the unbalanced deployment took 5% longer to execute compared to the balanced deployment.*

Figure 17 shows the percentage of errors each sampling strategy managed to capture for the balanced and unbalanced deployment. Tail based sampling managed to pick up all the errors, mixed sampling managed to pick up around 20% of the errors while head based sampling is at around five percent. These are around the same sampling percentage set at the daemonset collector. Figure 18 shows how much memory all the collected traces have taken. Tail based sampling has the most, next is head based sampling and at last mixed sampling. Head based sampling has 130% more stored compared to mixed sampling for both the balanced and unbalanced deployment. The

tail based sampling has collected 360% more compared to the mixed sampling for both the deployments.

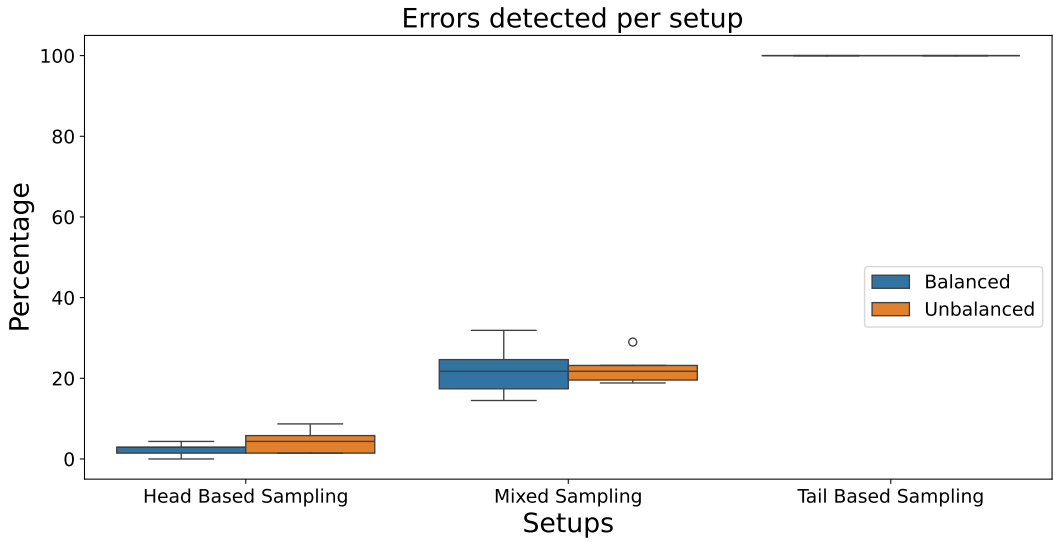

Figure 17: *The percentage of errors detected for each sampling strategy, for both balanced and unbalanced deployment in a two node cluster. Tail based sampling picked 100%, while mixed sampling picked up around 20% and head based sampling around 5%.*

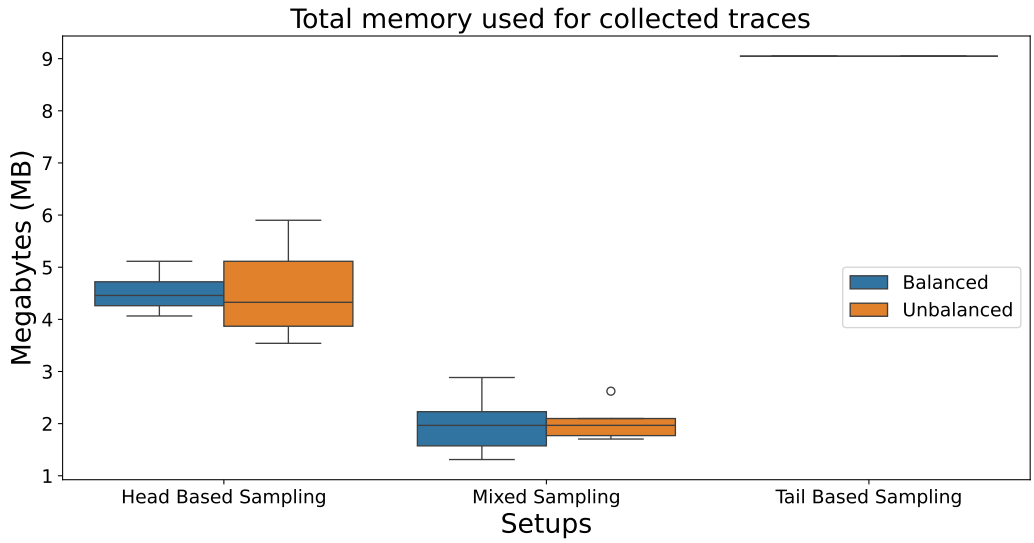

Figure 18: *Total memory consumed of all traces collected for each sampling strategy, for both balanced and unbalanced deployment in a two node cluster. Head based sampling has collected 130% more compared to mixed sampling, and tail based sampling has collected 360% more than the mixed sampling method.*

Figures 19 and 20 show the CPU work seconds of the collectors and Jaeger per node, for the balanced and unbalanced deployment. In both of the figures, tail based

sampling has taken the most time, followed by mixed sampling and at last head based sampling, which is slightly less than mixed sampling strategy. For both the balanced and unbalanced deployment, mixed sampling had 10% more CPU work seconds in total compared to head based sampling. Tail based sampling had 74% more work seconds in total in the balanced deployment, and 73% in the unbalanced deployment, compared to head based sampling.

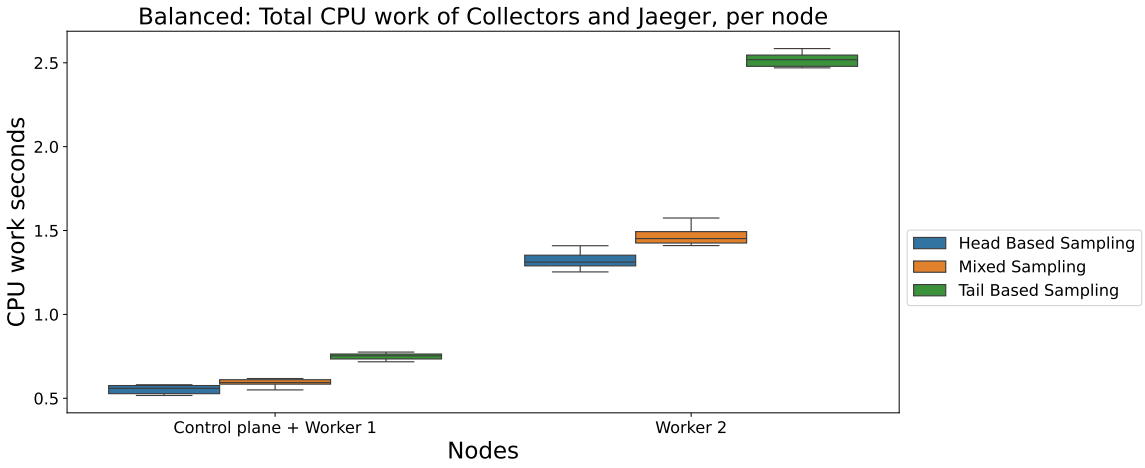

Figure 19: *CPU work seconds taken for the collectors and Jaeger to handle all requests in a balanced deployment, for each sampling strategy. In worker 1, mixed sampling had 7.5% more and tail based sampling had 35.7% more than head based sampling. In worker 2, mixed had 11% more, and tail based sampling had 71.5% more than head based sampling.*

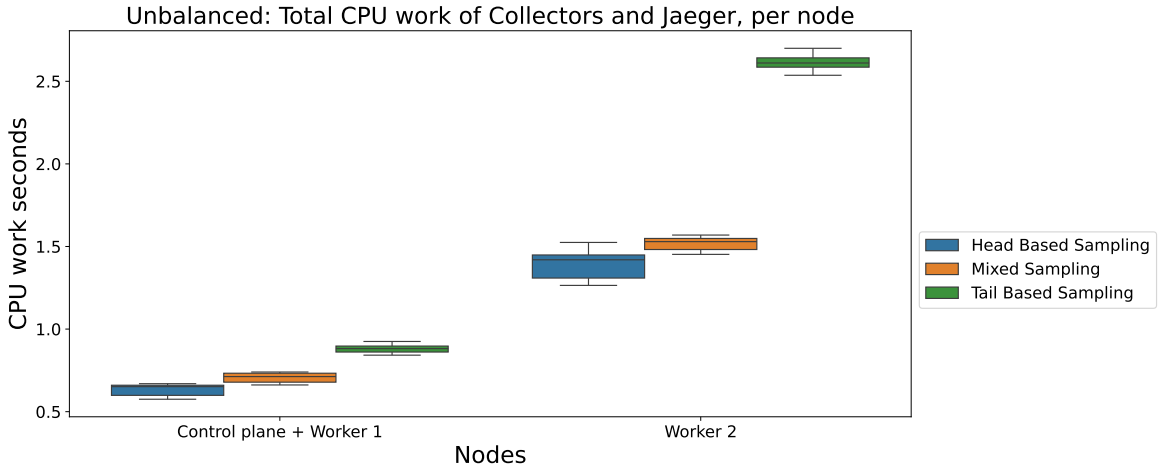

Figure 20: *CPU work seconds taken for the collectors and Jaeger to handle all requests in an unbalanced deployment, for each sampling strategy. In worker 1, mixed sampling had 11.8% more, and tail based sampling had 39.7% more than head based sampling. In worker 2, mixed had 9.1% more and tail based sampling had 88.2% more than head based sampling.*

Figures 21 and 22 show the memory usage of every sampling strategy for both balanced and unbalanced deployment. Similar trends as the CPU work seconds can be observed for the memory as well. Mixed sampling had around 9% more memory usage in total, compared to head based sampling for balanced and unbalanced deployment. Tail based sampling had 29% more in total in the balanced deployment and 31% in the unbalanced deployment compared to head based sampling.

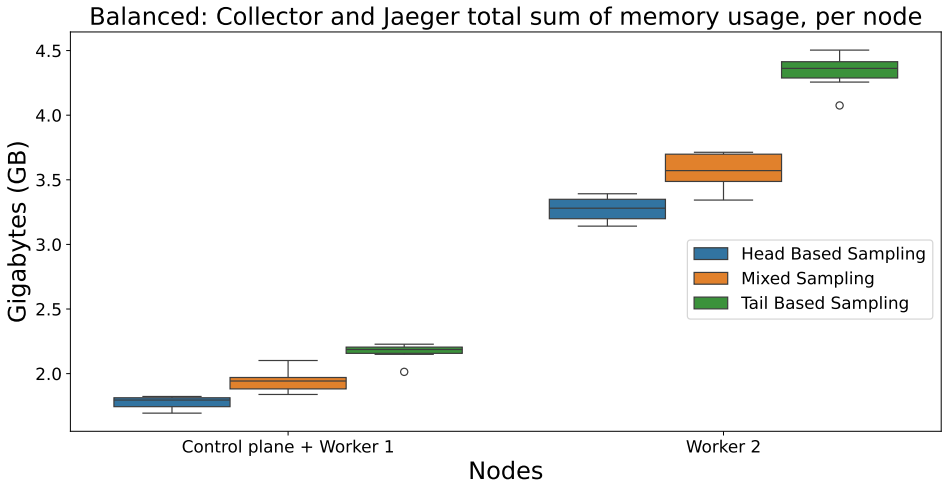

Figure 21: *Sum of memory used for the collectors and Jaeger to handle all requests in a balanced deployment, for each sampling strategy. In worker 1, mixed sampling had 9.3% more and tail based sampling had 21.9% more memory usage than head based sampling. In worker 2, mixed sampling had 9.1% more and tail based sampling had 32.5% more memory usage than head based sampling.*

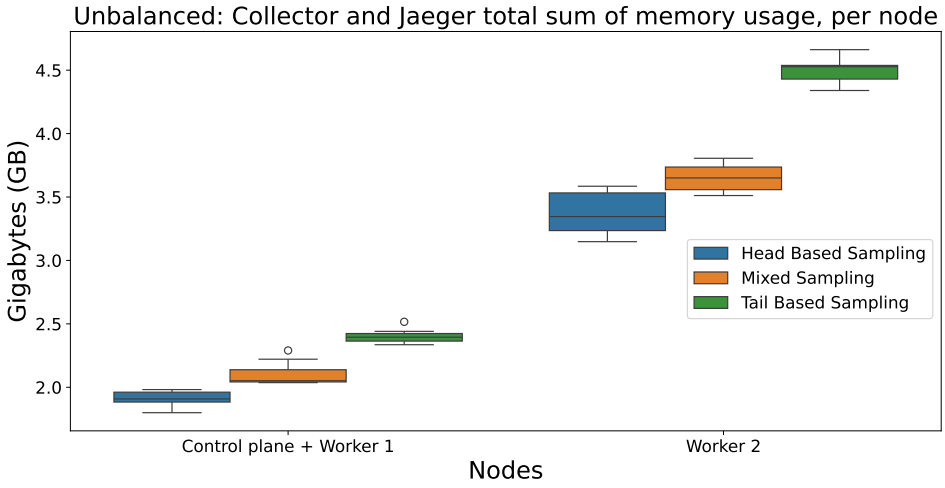

Figure 22: *Sum of memory used for the collectors and Jaeger to handle all requests in an unbalanced deployment, for each sampling strategy. In worker 1, mixed sampling had 10.2% more and tail based sampling had 25.7% more memory usage compared to head based sampling. In worker 2, mixed had 8.2% more and tail based had 32.2% more memory usage than head based sampling.*

Figure 23 and 24 show how much network traffic has been received to a node for the balanced and unbalanced deployment. In both cases, tail based sampling has received the most data from other nodes to its node. In Figure 23 head based sampling has a bit more received data to its node on worker 2 compared to the mixed sampling. The difference becomes slim in the unbalanced deployment seen in Figure 24 for worker 2, but there is a larger gap between worker 1 and 2. In total, mixed sampling had 0.6% more received bytes for the balanced deployment and 0.3% more for the unbalanced deployment compared to head based sampling. Tail based sampling had 5.6% more received bytes in the balanced deployment and 2.8% more in the unbalanced deployment, compared to head based sampling.

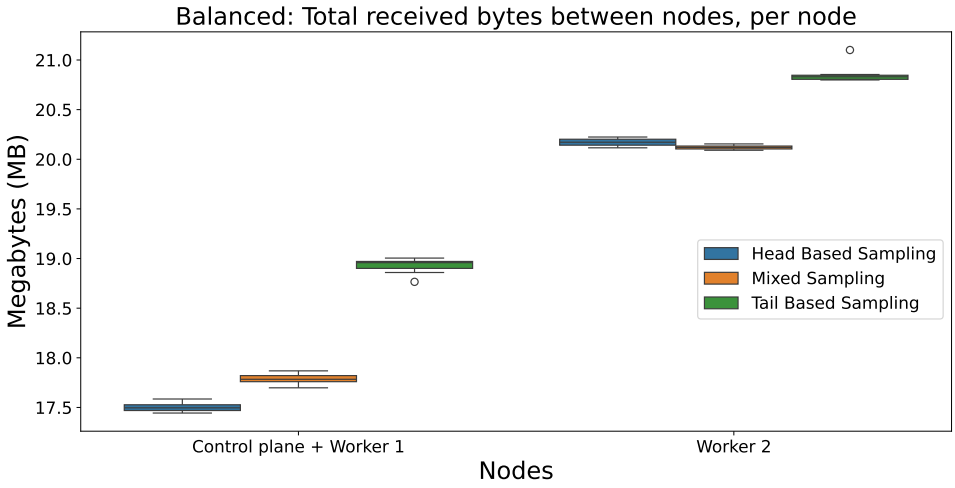

Figure 23: *Total received bytes of a node in a balanced deployment for different sampling strategies. In worker 1, mixed sampling had 1.6% more and tail based sampling had 8.1% more received bytes than head based sampling. In worker 2, head based sampling received 0.3% more and tail based sampling received 3.7% more than mixed sampling.*

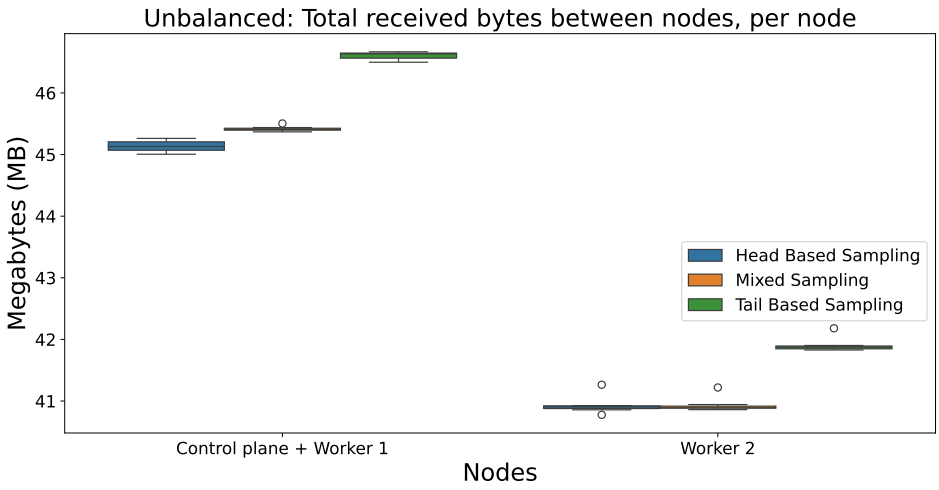

Figure 24: *Total received bytes of a node in an unbalanced deployment for different sampling strategies. In worker 1, mixed sampling received 0.6% more and tail based sampling received 3.3% more bytes than head based sampling. In worker 2, tail based sampling received 2.4% more bytes than mixed and head based sampling.*

5.2 Four Node Setup

Figure 25 shows the total time taken to get response for all the messages sent by the user, for all the sampling strategies in an unbalanced and balanced deployment. The differences between the sampling strategies are marginal, but the greatest difference can be observed between a balanced and unbalanced deployment.

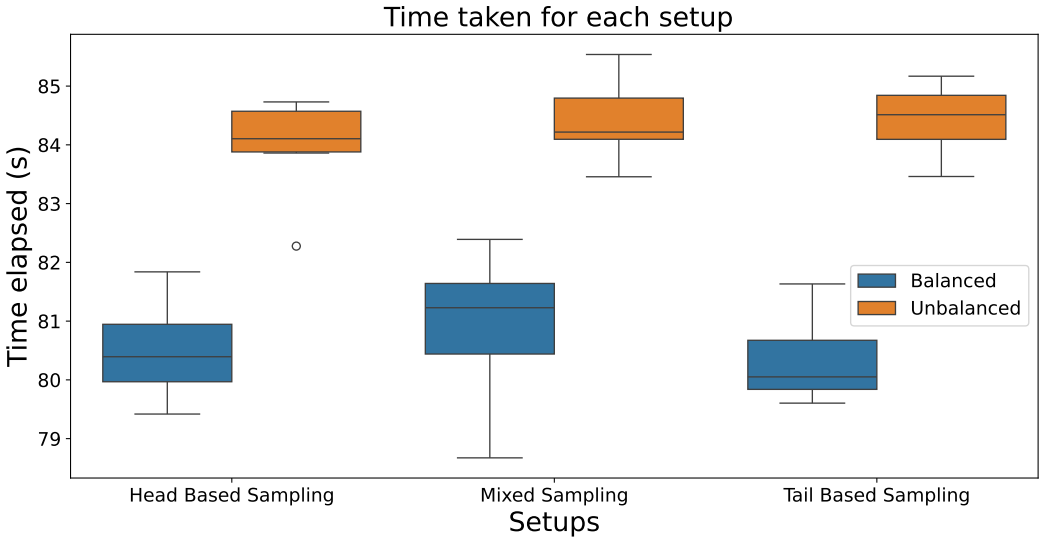

Figure 25: *Time taken for four nodes to handle all user sent requests for balanced and unbalanced setup, for each sampling strategy. Unbalanced deployment took 4.4% longer to run compared to the balanced deployment for head based sampling. The unbalanced deployment took 4.3% longer for mixed sampling and 5.1% longer for tail based sampling compared to the balanced deployment.*

Tail based sampling has managed to collect all errors that were sent in the system. Mixed sampling managed to pick up around 20% while head based sampling is at around five percent, as seen in Figure 26, which are around the same sampling values set for the sampling methods. Tail based sampling has collected most traces, and therefore has the most memory used for collected traces, seen in Figure 27. Head based sampling collected 126.5% more traces and tail based sampling collected 394.2% more traces compared to mixed sampling.

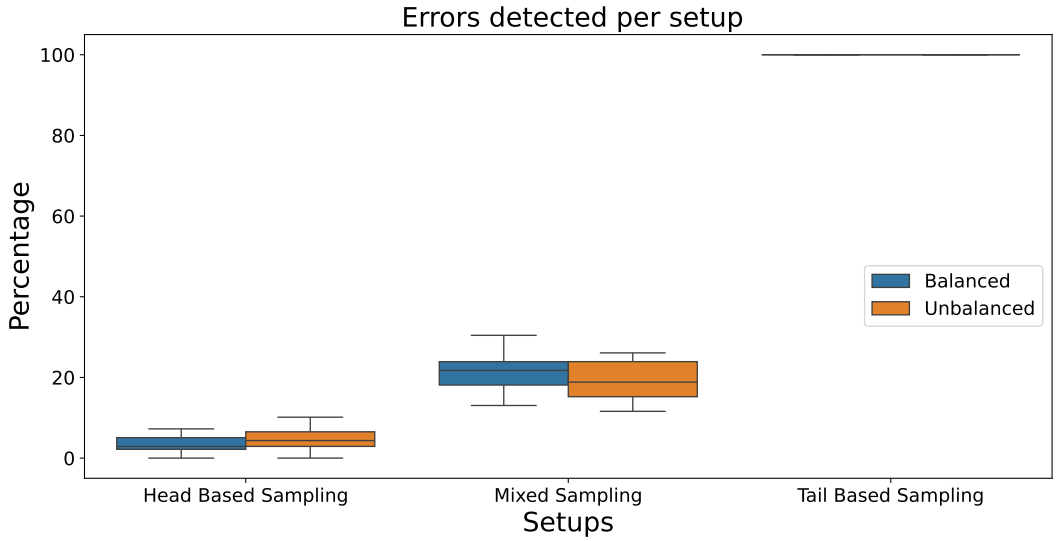

Figure 26: *The percentage of errors detected for each sampling strategy, for both balanced and unbalanced deployment in a four node cluster. Tail based sampling found all errors, mixed sampling found around 20% and head based sampling 5%.*

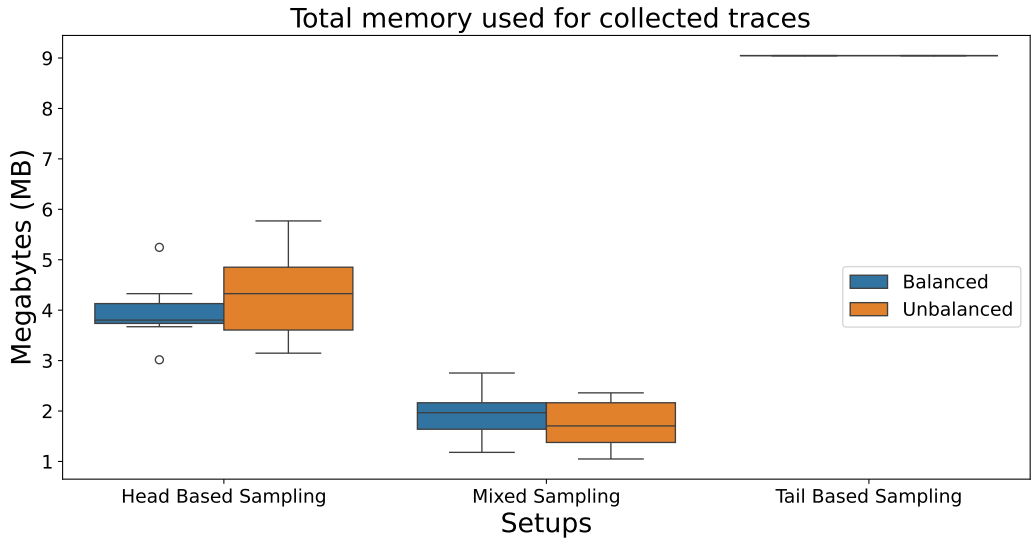

Figure 27: *Total memory consumed of all traces collected for each sampling strategy, for both balanced and unbalanced deployment in a four node cluster. Head based sampling collected around 126.5% more and tail based sampling 394.2% more traces compared to mixed sampling.*

Figure 28 shows the CPU work seconds of collectors and Jaeger, for a balanced deployment. In all cases, except in worker 1, tail based sampling takes the most seconds to handle all the requests. The differences between the sampling methods in worker 1 is slim, where worker 1 in general has the lowest CPU work seconds. Mixed sampling takes the second-longest time in worker 2 and 4, where head based sampling is ahead of mixed sampling in worker 3. Tail based sampling took in total 69.8% longer than head based sampling, and 50.4% longer than mixed sampling for the balanced deployment. Figure 29 shows the CPU work seconds for collectors and Jaeger for an unbalanced deployment. It has similar trends for all the sampling methods as the balanced deployment. Tail based sampling has the most CPU work for the balanced and unbalanced deployment in worker 2. In total, tail based sampling took 52.6% longer than mixed sampling and 68.5% longer than head based sampling.

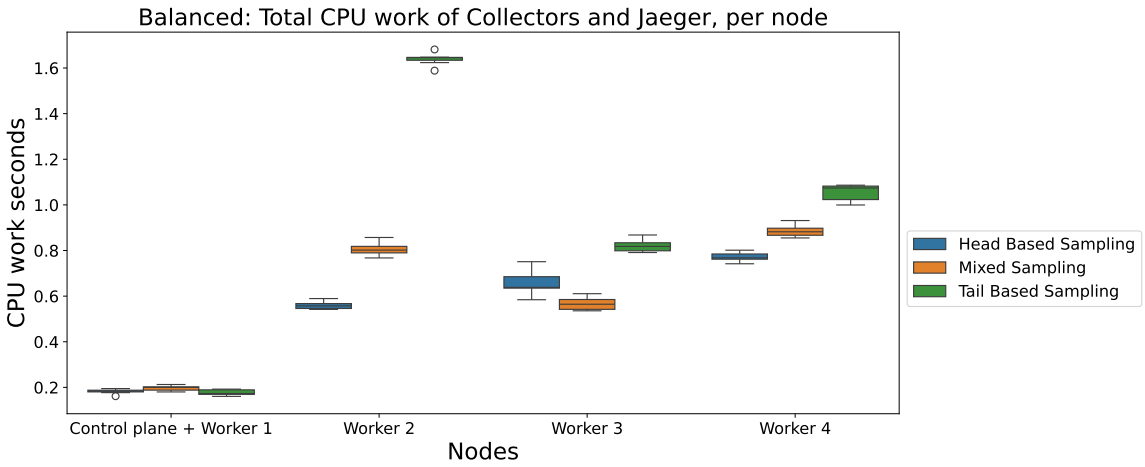

Figure 28: *CPU work seconds taken for the collectors and Jaeger to handle all requests in a balanced deployment, for each sampling strategy. In worker 1, mixed sampling had 7.7% more than head based sampling and 9.4% more than tail based sampling. In worker 2, tail based had 193.1% more than head based sampling and 103.4% more than mixed sampling. In worker 3, tail based sampling had 44.8% more than mixed sampling and 24.5% more than head based sampling. In worker 4, tail based sampling had 36.4% more than head based sampling and 18.9% more than mixed sampling.*

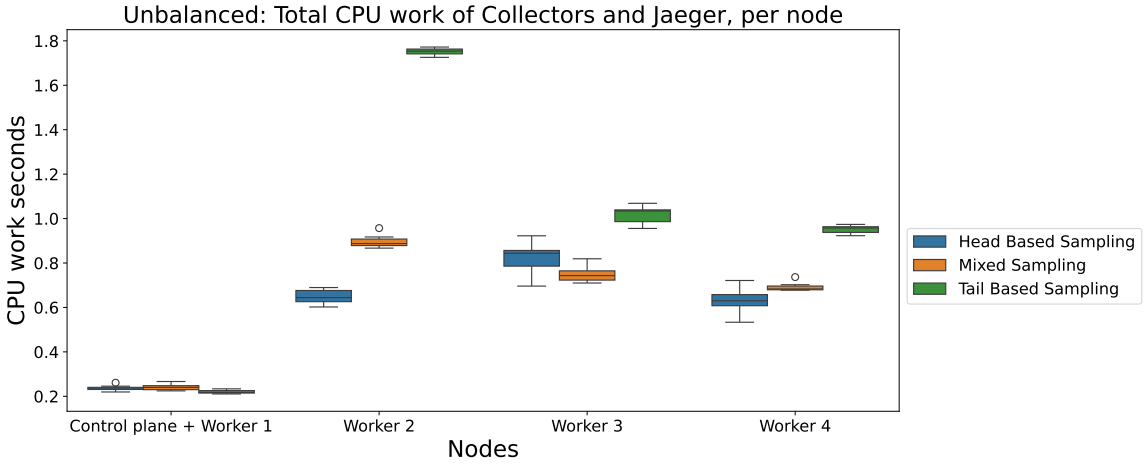

Figure 29: *CPU work seconds taken for the collectors and Jaeger to handle all requests in an unbalanced deployment, for each sampling strategy. In worker 1, mixed sampling had 1.9% more than head based sampling and 8.5% more than tail based sampling. In worker 2, tail based sampling had 170% more than head based sampling and 95.1% more than mixed sampling. In worker 3, tail based sampling had 35.5% more than mixed sampling and 23.7% more than had based sampling. In worker 4, tail based sampling had 50.7% more than head based sampling and 37.3% more than mixed sampling.*

Figure 30 shows the sum of the memory used of every time interval for a balanced deployment. In all the cases, tail based sampling has the most memory used, except

worker 1. Mixed sampling has the second most used memory, where the difference is slim in worker 1 compared to head based sampling and marginal in worker 3. Tail based sampling had in total 18.6% more memory usage than head based sampling and 10.4% more than mixed sampling, for the balanced deployment. Figure 31 shows similar results for the unbalanced deployment. One difference is that head based sampling has consumed more memory in worker 3 compared to mixed sampling. Tail based sampling had 16.2% more memory usage than head based sampling and 11.1% more than mixed sampling.

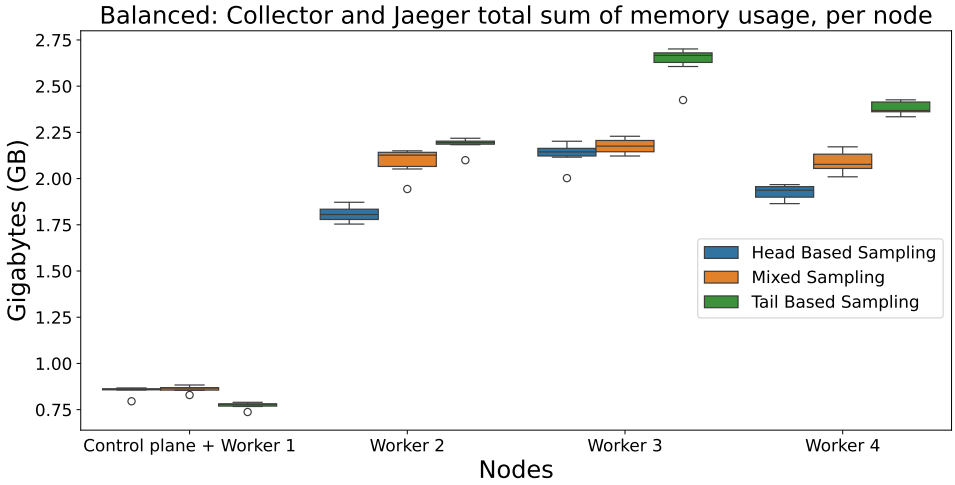

Figure 30: *Sum of memory used for the collectors and Jaeger to handle all requests in a balanced deployment, for each sampling strategy. In worker 1, mixed sampling had 1.1% more than head based sampling and 10.3% more than tail based sampling. In worker 2, tail based sampling had 4.5% more than mixed sampling and 20.8% more than head based sampling. In worker 3, tail based sampling had 23.4% more than head based sampling and 20.9% more than mixed sampling. In worker 4, tail based sampling had 23.7% more than head based sampling and 14% more than mixed sampling.*

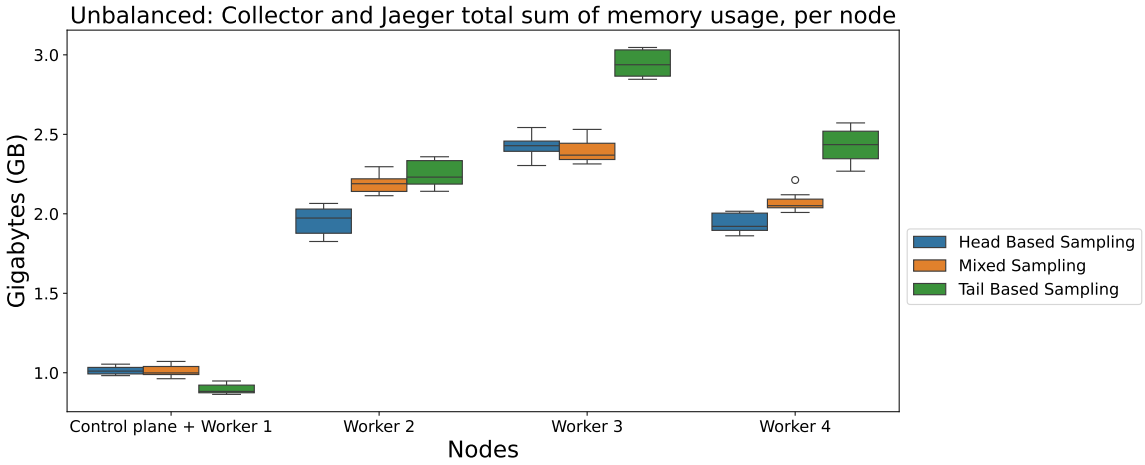

Figure 31: *Sum of memory used for the collectors and Jaeger to handle all requests in an unbalanced deployment, for each sampling strategy. In worker 1, mixed and head based sampling had around 11.3% more than tail based sampling. In worker 2, tail based sampling had 15.3% more than head based sampling and 3% more than mixed sampling. In worker 3, Tail based sampling had 21.5% more than head based sampling and 22.9% more than mixed sampling. In worker 4, tail based sampling had 25.1% more than head based sampling and 17% more than mixed sampling.*

Figure 32 shows the total received network data per node for the balanced deployment. The sampling strategies have received similar amounts of bytes in worker 1. Worker 1 and 2 have received the most networking traffic and tail based sampling has received the most compared to other sampling methods. Tail based sampling has received 8.9% more bytes than head based sampling and 7.1% more than mixed sampling. For the unbalanced deployment, worker 3 has received the most network traffic and the sampling methods have received similar amounts of bytes in worker 1, seen in Figure 33. Tail based sampling has 5.1% more received bytes than head based sampling and 4.2% more than mixed sampling.

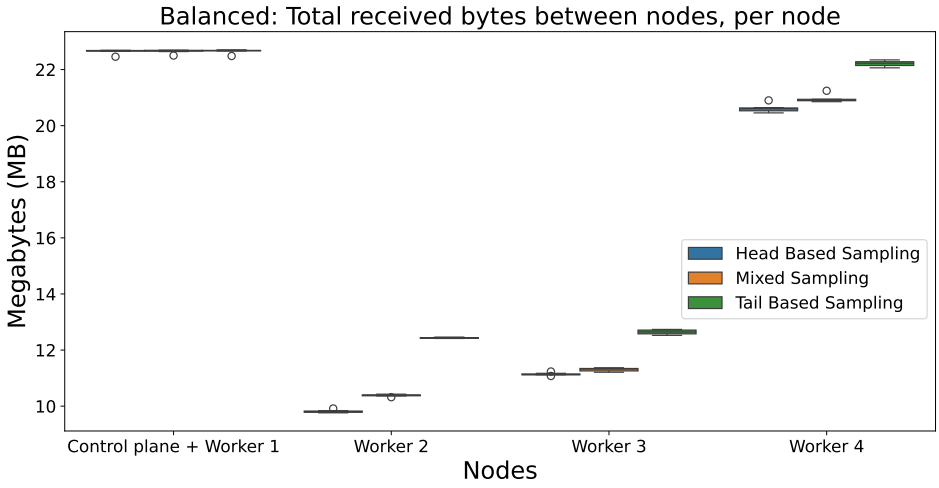

Figure 32: *Total received bytes of the node in a balanced deployment for different sampling strategies. In worker 1, the sampling methods had similar amounts received. In worker 2, tail based sampling had 26.7% more than head based sampling and 19.7% more than mixed sampling. In worker 3, tail based sampling had 13.5% more than head based sampling and 11.9% more than mixed sampling. In worker 4, tail based sampling had 7.8% more than head based sampling and 6% more than mixed sampling.*

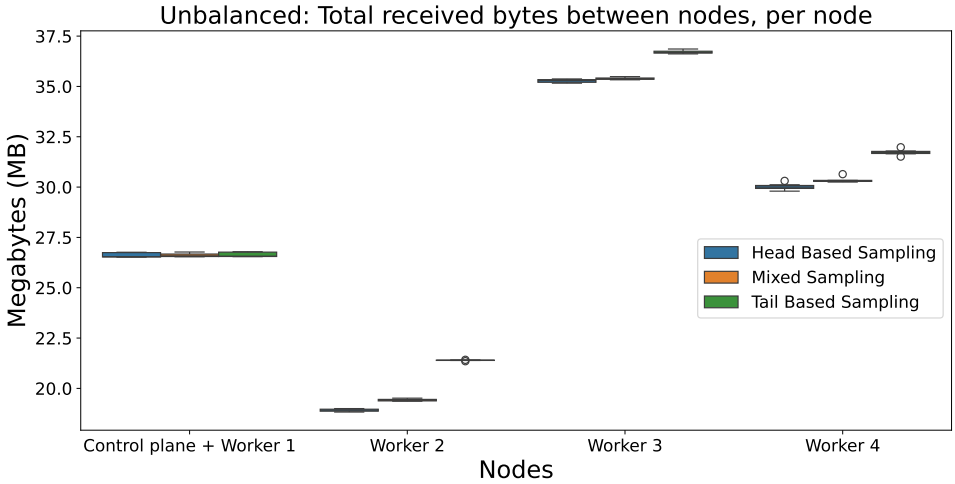

Figure 33: *Total received bytes of the node in an unbalanced deployment for different sampling strategies. In worker 1, the sampling methods had similar amounts received. In worker 2, tail based sampling had 13.1% more than head based sampling and 10.1% more than mixed sampling. In worker 3, tail based sampling had 4.1% more than head based sampling and 3.7% more than mixed sampling. In worker 4, tail based sampling had 5.7% more than head based sampling and 4.5% more than mixed sampling.*

6 Discussion

This chapter discusses the results presented in Chapter 5. The two and four node setup is discussed for the different sampling strategies, and later the two and four nodes are compared to each other. At last, the challenges of this project are discussed to give a better insight into the difficulties of cloud projects and OpenTelemetry.

6.1 Two Node Setup

When it comes to the time taken of the experiment, there are only small differences between the sampling strategies, as seen in Figure 16. The reason for this is that the call chain app has a much higher burden on the system compared to distributed tracing. The call chain app sends HTTP requests between each other and to the daemonset collectors, which has a much greater impact compared to the gRPC messages sent between collectors and the sampling that takes place in the collectors. This in turn makes the request throughput between the sampling strategies much smaller. A much greater difference can be observed between the balanced and unbalanced setup. The reason is that the overhead of starting a connection between the nodes has a much greater impact, rather than just sending a message between processes in the same node. One aspect to note is that the difference would have been much greater if there had been more noise on the computer that performed the experiments, such as overutilisation of the computer. This could be common in computer centers hosting VMs. Another aspect to take into consideration is that if there would be more network traffic noise, the difference would have increased even more. If the nodes had been placed in different countries, the difference between the balanced and unbalanced runs would have grown, as it would have higher latency and noise in the network.

The error detection seen in Figure 17 is as expected. Head based sampling finds around five percent, mixed is at around 20%, and tail finds all the errors in the system. Since the tail based setup goes through every span sent in the system, it manages to sample through all errors. Figure 18 shows also an expected result. Tail based sampling is at the top since it sampled ten percent of all the messages since ten percent of the messages sent contained errors. Head based sampling collected the second most traces and mixed sampling had the least collected traces. This is because mixed sampling samples 20% in the daemonset collector and later only collects the errors, which are at ten percent. This puts the total collected traces at around two percent, which is lower than the head based sampling setup, which had a sampling set to five percent.

Tail based sampling had more CPU work compared to mixed and head based sampling for both the balanced and unbalanced deployment, seen in Figures 19 and 20. This is because tail based sampling handles more messages sent between collectors compared to the two other sampling methods. A much greater difference can be observed between workers 1 and 2 since the loadbalancer and Jaeger are deployed in node 2. This in turn puts more burden on worker 2 compared to worker 1. The daemonset also receives almost four times more spans in node 2 compared to node 1. Comparing the balanced and unbalanced deployment, the trend is similar, since the distribution of spans sent to collectors is not too different.

The total sum of memory used by the collectors and Jaeger follows the same trend as CPU usage, as seen in Figures 21 and 22. Tail based consumed the most memory, which makes sense since it also stored the most traces at the end. Mixed sampling takes second place, and that is because the memory consumed in the daemonset collector, loadbalancer, and collector outweighs the memory consumed by Jaeger, when compared against head based sampling in worker 2. Mixed sampling also consumed more in worker 1 as it lets through more spans in its daemonset collector compared to the head based sampling. When finally reaching the collector in worker 1, the collector has to allocate enough memory to handle every received span. Although it sends fewer traces after the collector has handled the spans, the memory needed to handle them is greater. Both balanced and unbalanced deployments follow a similar trend since the distribution of spans generated is not too far off.

When observing the total received bytes to the nodes for the balanced deployment, tail based sampling receives the most, as seen in Figure 23. This makes sense as no filtering happens in the daemonset collector and even when the filtering happens, there are still more traces collected compared to the mixed and head based sampling. When comparing mixed and head based sampling, it can be seen that head based sampling has more received data compared to the mixed sampling for worker 2, but it shifts in worker 1. This is because more traces are sent to the collector in worker 1 from the loadbalancer, for mixed sampling, as it has a higher sampling rate in the daemonset collector. Since there are fewer traces sent to Jaeger from the collector in worker 1 for the mixed sampling, it can be seen that head based sampling has received more data. In the unbalanced setup, worker 1 receives much more compared to worker 2, as seen in Figure 24. This is because the services send much more requests between nodes and worker 1 receives much more from a single request compared to

the balanced deployment. The sampling methods follow the same trends, but the difference between mixed and head based sampling is only marginal in worker 2 since sampling data becomes only a small part compared to the total amount of data sent between services.

6.2 Four Node Setup

As discussed in Section 6.1, the difference in time taken to handle the requests is minimal when comparing sampling strategies, but much greater when comparing balanced and unbalanced deployment for four nodes, seen in Figure 25. This is due to the low impact of distributed tracing on the system as a whole compared to the HTTP requests that the system has to handle. The impact is much greater when more messages are sent between nodes in the system. This comes from the overhead of sending and receiving messages between nodes, which is greater compared to sending messages within a node. One aspect to note is that worker 4 has 31 internal messages sent and receives 12 messages from other nodes in the balanced setup, and the overall system still manages requests quicker compared to the unbalanced setup. This shows that the overhead of sending messages between nodes is still more intense compared to having a worker do more work.

The system detects similar amounts of errors for the four node setup and the two node setup, seen in Figure 26. Since the same amount of requests were sent and the configurations of the different sampling strategies were the same, a similar outcome can be seen, as expected. This is also reflected in the total memory used for the collected traces, seen in Figure 27. Since the number of errors sent was at ten percent, tail based sampling is at the top, followed by head based sampling which had a sampling percentage set to five, and mixed sampling. Since the mixed sampling had a probabilistic sampling rate set to 20% in the daemonset collector and later only collected errors, which were ten percent, the total sample rate is once more at around two percent.

When it comes to the total CPU work seconds of the four node setup for the collectors, loadbalancer, and Jaeger, the balanced and unbalanced deployment have similar trends for the sampling strategies, seen in Figures 28 and 29. In worker 1, there is only a small difference between the sampling strategies, where tail based sampling seems to be just slightly lower for both the balanced and unbalanced deployment. This is because there is only a daemonset collector in worker 1 and the only job for the tail based sampling is to export the spans. Mixed and head based sampling has to hash the trace ID of

the span and later decide whether to export it or not, which when it comes to CPU work seconds, seems to be just slightly more work. Worker 1 also exports few traces compared to the other workers for the balanced and unbalanced deployment, which is also why the difference is marginal between the sampling strategies. A great difference between tail based sampling and the other two sampling strategies can be observed in worker 2, for both the balanced and unbalanced deployments. Worker 2 has a daemonset collector for both of the deployments, and the loadbalancer is deployed in worker 2. All the daemonset collectors send their spans to the loadbalancer, which forces the loadbalancer to handle all the spans. The spans have to be hashed and later all the spans for a trace have to be sent to the same collector. This takes the most computing resources for both the balanced and unbalanced deployment when it comes to the tail based sampling method. Since mixed and head based sampling sends fewer traces from the daemonset collector, fewer computational resources are needed later on in the chain. This could be a potential bottleneck which needs to be thought of when implementing OpenTelemetry in a cloud environment and tail based sampling is used. In worker 3, tail based sampling takes the most time CPU work seconds and is later followed by head based sampling. The fact that head based sampling takes more CPU work compared to mixed sampling can be explained by the number of samples that are sent and that Jaeger is deployed in worker 3. Mixed sampling receives more spans in the collector deployed in worker 3 but sends fewer messages to Jaeger, and Jaeger also receives fewer spans from the collector deployed in worker 4 for the head based sampling strategy. These factors make it so that head based sampling requires more resources compared to mixed sampling. At last, in worker 4 for balanced and unbalanced deployment, tail based sampling takes the most CPU work seconds since it sends all spans through the daemonset collector. It later receives around half of them back in the collector deployed in worker 4. The tail based method then samples errors in the collector and sends them to Jaeger, which takes more CPU work seconds compared to mixed and head based sampling. Mixed sampling is above head based sampling in worker 4 since it sends more span in the daemonset collector and later receives more in the collector deployed in worker 4. It can be seen that for all sampling methods, worker 4 takes more CPU work seconds in the balanced deployment compared to workers 1 and 3, but for the unbalanced deployment, it is the other way around. This is because 74 spans are sent to the daemonset collector in the balanced deployment, compared to 13 spans in worker 3. For the unbalanced deployment, 38 spans are sent to the daemonset collector in worker 3 and 34 spans in

worker 4. Although Jaeger receives all the spans in the end, it can be seen that worker 4 is more CPU intensive in the balanced deployment, since 74 spans outmatches it.

When it comes to the sum of memory usage for collectors, loadbalancer, and Jaeger, it can be observed that mixed and head based sampling take closely as much memory while tail based sampling takes less for worker 1, seen in Figures 30 and 31. The daemonset collector deployed in worker 1 receives the same amount of requests, but it has to hash on trace ID for mixed and head based sampling. Although tail based sampling sends more spans from the daemonset collector, it still requires less memory compared to the memory needed to hash the incoming spans. In worker 2 for balanced and unbalanced deployment, tail based sampling has the highest memory usage, followed by mixed sampling and at last head based sampling. This is because tail based sampling receives more spans since no messages get filtered in the daemonset collector, resulting in more received spans to the loadbalancer. One aspect to note is that the difference between the tail based sampling and the other sampling strategies were much more dramatic in CPU work seconds compared to memory usage. This indicates that the loadbalancer is more CPU intensive than memory intensive. In worker 3, tail based sampling had the most memory usage, whereas mixed and head based sampling were quite similar. Tail based sampling is expected to have the highest memory usage since more spans are sent to the collector in worker 3 and more spans are stored in Jaeger, compared to the other sampling strategies. Mixed and head based sampling is quite even since mixed sampling has to handle more spans in the collector, resulting in higher memory consumption, but sends fewer spans to Jaeger. At last, for both balanced and unbalanced deployment in worker 4, tail based sampling has the highest memory usage, followed by mixed sampling and at last head based sampling. This is because it receives the most spans to the collector, and mixed sampling receives the second most.

When it comes to total received bytes to nodes, tail based sampling has the most for all workers, followed by mixed sampling and at last head based sampling, seen in Figures 32 and 33. The difference between the sampling methods is slim in worker 1 for both the deployments since worker 1 receives few requests, no spans, and it has Prometheus deployed on the node, which gathers metrics from all nodes. Node 1 also host the control plane, which receives additional data. Since few messages are sent in the balanced deployment, worker 1 can be seen at the top, followed by worker 4 which receives the second most requests to the node. Since sending requests between services

using HTTP has higher overhead compared to gRPC that the collectors and load-balancer use, the impact is less. The difference becomes clearer when comparing the sampling methods. It is also important to note that sending requests also comes with additional networking overhead, since each sent message also comes with a response, which in turn increases the amount of data received to a node. This can be observed when comparing workers 3 and 4 for the unbalanced deployment seen in Figure 33. Although worker 3 only receives 13 requests, compared to worker 4s 34 requests, it transmits 25 requests, whereas worker 4 transmits none. It also receives spans from worker 4 from its collector sent to Jaeger deployed in worker 3. This in total puts worker 3 above worker 4 when measuring total received bytes.

6.3 Node Comparison

It is important to consider the placement of services and collectors when increasing or decreasing the number of nodes in a system. The result for both the 2 and 4 node systems, when looking at the total time taken, showed that strategically placing services that communicate often has a great impact on how many requests the system can get through in a shorter time. The result could vary even more if there is an increase in network traffic noise in the system. When increasing the number of nodes, one has to consider potential bottlenecks in the system. For instance, regarding CPU work seconds for the 4 node setup the increase of resources the loadbalancer took compared to the rest of the system was shown. This could be a potential bottleneck if there would suddenly be a spike in traffic in the system. Tail based sampling had the most memory usage in worker 3 for the 4 node setup, in both the balanced and unbalanced deployment. This was due to Jaeger being placed there and also a collector. One has to then consider where to deploy services and tools, and consider where a node might have more memory resources and less CPU resources, but also the other way around. This is one of the challenges that has to be considered when moving services to cloud-based systems. It is also important to consider the total impact of sampling compared to the system as a whole. In Figure 32 the difference between tail and head based sampling was almost 9%, when looking at the total received bytes to each node. If a more intense system sent more data between nodes, this difference is expected to be less, as sampling would have less impact compared to the total volume sent.

6.4 Challenges

When initially starting this project, Cloud Guru servers were used to perform experiments. Cloud Guru is an online service that provides VMs and tools for cloud projects, where business licences were provided by Nasdaq for this project. The results were relatively stable at first, but later on started to vary greatly between experiments. This was due to overutilisation of the computers that the VMs were running on. Steal time, which is the CPU time taken away from a VM, could vary between 5 to over 800 seconds for a single experiment. This factor has to be considered when deploying services to the cloud, where one loses control of the hardware.

Another challenge with this project was integrating OpenTelemetry into the system. The system is in constant change and the behaviours could change as time goes by. The loadbalancer needed placeholder values in its configuration to be able to function correctly, which is a strong indication of its work in progress. Another aspect to consider is auto-instrumentation for different programming languages. The call chain application built for this experiment was written in Python and was injected with an HTTP exporting mechanism, since gRPC is not supported at the time of writing. Some programming languages support gRPC, which could change the outcome of the experiments.

7 Conclusion

The greatest impact on the system is observable when few requests are sent between nodes, in terms of request throughput. The sampling strategies had a low impact on the system as a whole, but when comparing the sampling strategies to each other by measuring how many resources the collectors, loadbalancer, and Jaeger took, tail based sampling performed the worst. It had on average 71.33% CPU, 23.7% memory and 5.6% network overhead compared to head based sampling. This was at the cost of effective error detection, where it managed to pick up 100% of the errors. Tail based sampling was followed by mixed sampling, which had on average 10.8% CPU, 7.5% memory and 0.9% network overhead compared to head based sampling. One aspect to note is that for the 4 node system, head based sampling performed worse than mixed sampling where head based sampling required more resources in a worker node with Jaeger, since it had a higher sampling rate. Head based sampling had the worst error detection but required overall fewer resources. It is therefore recommended to use tail based sampling when attribute detection is of highest importance, such as error detection. If running a performance sensitive application, head based sampling is a recommended strategy, as it lowers the overall overhead of distributed tracing. Mixed sampling is recommended when it is important to lower the overall overhead and memory for storing traces is an issue, as it only keeps traces of interest. It can also lower some overhead compared to tail based sampling, as it on the first instance of receiving a span samples to later on lower the resources required.

7.1 Future Work

The experiments used the auto-instrumentation tool to send spans from the services to the collectors. It would be of interest to see how the behaviour might change when doing manual instrumentation. One could then also explore how different exporting protocols other than OTLP affects the system. Another way to extend the work is to see how metric and log collection could impact the system, in comparison to distributed tracing. Sampling methods could also be used for logs and metric collection to see how the overhead might change for different sampling techniques. A third way of extending this work is by increasing the load of the experiments, in a less limited work environment, to get a better understanding of how the system scales. At last, using artificial intelligence to predict system errors based on metrics from the services and only then sample traces could be of interest to see the overall performance impact on cloud systems.

References

Bento, Andre et al. (2021). "Automated analysis of distributed tracing: Challenges and research directions". In: *Journal of Grid Computing* 19, pp. 1–15.

Ertl, Otmar (2021). "Estimation from Partially Sampled Distributed Traces". In: *arXiv preprint arXiv:2107.07703*.

Nasdaq (2023). *About Nasdaq*. English. URL: `https://www.nasdaq.com/about` (visited on 04/08/2024).

Newman, Sam (2021). *Building microservices*. " O'Reilly Media, Inc.".

Niedermaier, Sina et al. (2019). "On observability and monitoring of distributed systems–an industry interview study". In: *Service-Oriented Computing: 17th International Conference, ICSOC 2019, Toulouse, France, October 28–31, 2019, Proceedings 17*. Springer, pp. 36–52.

OpenTelemetry Authors (2023). *OpenTelemetry Operator for Kubernetes*. English. URL: `https://opentelemetry.io/docs/concepts/sampling/` (visited on 03/01/2024).

— (2024a). *Code-based*. English. URL: `https://opentelemetry.io/docs/concepts/instrumentation/code-based/` (visited on 02/29/2024).

— (2024b). *Components*. English. URL: `https://opentelemetry.io/docs/concepts/components/` (visited on 02/29/2024).

— (2024c). *Configuration*. English. URL: `https://opentelemetry.io/docs/collector/configuration/` (visited on 02/29/2024).

— (2024d). *OpenTelemetry Operator for Kubernetes*. English. URL: `https://opentelemetry.io/docs/kubernetes/operator/` (visited on 02/29/2024).

— (2024e). *What is OpenTelemetry?* English. URL: `https://opentelemetry.io/docs/what-is-opentelemetry/` (visited on 02/29/2024).

— (2024f). *Zero-code*. English. URL: `https://opentelemetry.io/docs/concepts/instrumentation/zero-code/` (visited on 02/29/2024).

OpenTelemetry Github Contributors (2024). *OpenTelemetry Collector Contrib*. English. URL: `https://github.com/open-telemetry/opentelemetry-collector-contrib/tree/main/processor/tailsamplingprocessor` (visited on 03/03/2024).

Pahl, Claus et al. (2017). "Cloud container technologies: a state-of-the-art review". In: *IEEE Transactions on Cloud Computing* 7.3, pp. 677–692.

Picoreti, Rodolfo et al. (2018). "Multilevel observability in cloud orchestration". In: *2018 IEEE 16th Intl Conf on Dependable, Autonomic and Secure Computing, 16th Intl Conf on Pervasive Intelligence and Computing, 4th Intl Conf on Big Data In-*

telligence and Computing and Cyber Science and Technology Congress (DASC/PiCom/DataCom/CyberSciTech). IEEE, pp. 776–784.

Red Hat (2020). *What is Kubernetes?* English. URL: `https://www.redhat.com/en/topics/containers/what-is-kubernetes` (visited on 02/27/2024).

Reichelt, David Georg, Stefan Kühne, and Wilhelm Hasselbring (2021). "Overhead comparison of opentelemetry, inspectit and kieker". In.

Siddiqui, Tamanna, Shadab Alam Siddiqui, and Najeeb Ahmad Khan (2019). "Comprehensive analysis of container technology". In: *2019 4th International Conference on Information Systems and Computer Networks (ISCON)*. IEEE, pp. 218–223.

Sridharan, Cindy (2018). *Distributed systems observability: a guide to building robust systems*. O'Reilly Media.

The Kubernetes Authors (2021). *Glossary*. English. URL: `https://kubernetes.io/docs/reference/glossary/` (visited on 02/28/2024).

— (2023). *Container Runtime Interface (CRI)*. English. URL: `https://kubernetes.io/docs/concepts/architecture/cri/` (visited on 02/28/2024).

— (2024). *Kubernetes Components*. English. URL: `https://kubernetes.io/docs/concepts/overview/components/` (visited on 02/27/2024).

Thönes, Johannes (2015). "Microservices". In: *IEEE software* 32.1, pp. 116–116.

Van Steen, Maarten and Andrew S Tanenbaum (2017). *Distributed systems*. Maarten van Steen Leiden, The Netherlands.

Yu, Xiao et al. (2016). "Cloudseer: Workflow monitoring of cloud infrastructures via interleaved logs". In: *ACM SIGARCH Computer Architecture News* 44.2, pp. 489–502.

Zhang, Lei et al. (2023). "The Benefit of Hindsight: Tracing {Edge-Cases} in Distributed Systems". In: *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pp. 321–339.

Zhang, Qi et al. (2018). "A comparative study of containers and virtual machines in big data environment". In: *2018 IEEE 11th International Conference on Cloud Computing (CLOUD)*. IEEE, pp. 178–185.